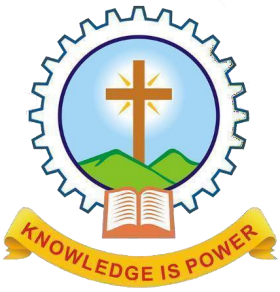
MAR ATHANASIUS COLLEGE OF ENGINEERING
KOTHAMANGALAM
(Government Aided Autonomous Institution)



Department of Computer Applications

Mini Project Report

REAL-TIME NETWORK INTRUSION DETECTION AND ALERT SYSTEM

Done by

ABHINAV SAJEEV

Reg No: MAC25MCA-2002

Under the guidance of

Dr. Sonia Abraham

2025–2027

MAR ATHANASIUS COLLEGE OF ENGINEERING

KOTHAMANGALAM

(Government Aided Autonomous Institution)

CERTIFICATE

Real-Time Network Intrusion Detection and Alert System

Certified that this is the bonafide record of project work done by

ABHINAV SAJEEV

Reg No: MAC25MCA-2002

during the third semester, in partial fulfilment of requirements for award of the degree,

Master of Computer Applications

of

APJ Abdul Kalam Technological University, Thiruvananthapuram

Dr. Sonia Abraham
Faculty Guide

Dr. Sonia Abraham
Head of the Department

Dr. Manu John
Project Coordinator

Internal Examiners

Acknowledgement

First and foremost, I express my gratitude to God Almighty for showering blessings and granting the strength, perseverance, and wisdom to complete this mini-project work successfully.

I express my profound gratitude and heartfelt thanks to Dr. Sonia Abraham, Head of the Department of Computer Applications, Mar Athanasius College of Engineering, Kothamangalam, for providing excellent academic facilities, lab resources, and continuous encouragement throughout this endeavour.

I extend my deep gratitude and sincere appreciation to my project guide, Dr. Sonia Abraham, for invaluable guidance, constructive suggestions, insightful feedback, and patient support rendered during all stages of this project development.

I express my sincere gratitude to the Project Coordinators, Dr. Manu John and Prof. Minu Maria Joseph, as well as the faculty members and technical staff of the Department of Computer Applications, for their kind cooperation, valuable guidance, and assistance.

Finally, I express my deepest appreciation to my parents, family members, and friends for their continuous moral support, understanding, and encouragement throughout the completion of this project.

Abhinav Sajeev
MAC25MCA-2002

Abstract

Network security has become increasingly critical as modern cyberattacks grow in frequency, stealth, and complexity, threatening organizational digital infrastructure across all sectors. Traditional signature-based intrusion detection systems struggle to detect novel zero-day exploits, evasive low-rate probing, and evolving volumetric attack vectors. Machine learning offers a promising paradigm for adaptive anomaly detection, yet operational implementations frequently encounter significant challenges, including severe class imbalance between benign and malicious flows, prohibitive inference latency, and the absence of end-to-end integration between theoretical offline models and live packet capture pipelines.

This work presents the Real-Time Network Intrusion Detection and Alert System (NIDAS), an end-to-end machine learning platform engineered for continuous network monitoring and automated threat alerting. The system is trained and evaluated on a curated corpus of 1,799,905 network flows derived from the CIC-IDS2017 benchmark, covering Normal Traffic alongside four prevalent attack vectors: Denial of Service (DoS), Distributed Denial of Service (DDoS), Port Scanning, and Brute Force. A rigorous five-stage feature selection methodology, comprising zero-variance removal, collinearity reduction ($r > 0.90$), Kruskal-Wallis non-parametric significance testing, Random Forest feature importance scoring, and domain validation, prunes 77 raw flow features down to an optimal 40-feature operational space.

To overcome the operational limitations of single-model classifiers, NIDAS employs a soft-voting ensemble coupling Random Forest and Extreme Gradient Boosting (XGBoost). Random Forest ($B = 200$ trees) provides variance dampening across noisy packet metrics, while XGBoost (300 estimators with histogram-based binning) supplies precise bias reduction for difficult boundary distinctions. Balanced sample weighting counters class imbalance without introducing synthetic oversampling artifacts. On an independent test set of 394,168 flows, the ensemble achieves 99.88% overall accuracy, a macro-averaged F1-score of 99.76%, and sub-millisecond per-flow inference latency (< 0.6 ms).

The operational pipeline is deployed as a lightweight client-server application featuring asynchronous live packet capture via Npcap/libpcap, online feature extraction using Welford's algorithm, and a stateful cross-flow sliding window detector for stealthy port scans. Inferred threats are categorized into color-coded severity tiers (Critical, High, Medium) and streamed via FastAPI WebSockets to an interactive React SOC dashboard with automated self-traffic suppression. The result is a robust, production-ready solution that bridges theoretical intrusion detection with low-overhead, real-time security operations.

List of Tables

2.1 Summary of Paper 1 3
2.2 Summary of Paper 2 5
2.3 Summary of Paper 3 7
2.4 Literature Summary 8

3.1 Flow Volume and Packet Length Statistics (Features 1–10) 15
3.2 Flow Throughput and Inter-Arrival Timing Dynamics (Features 11–20) 16
3.3 Directional Timing and Protocol Header Attributes (Features 21–30) 17
3.4 TCP Control Flags, Windowing, and State Dynamics (Features 31–40) 18
3.5 Software Environment 38
3.6 Hardware Environment 39

4.1 Empirical Model Evolution and Validation Performance across Milestones . . . 46

5.1 Model Comparison on Test Set (394,168 Flows) 48
5.2 Per-Class Evaluation Metrics - Ensemble (Test Set) 50
5.3 TP, FP, FN, TN, FPR and FNR per Class - Ensemble (Test Set) 51

10.1 Minimum Software Requirements 64
10.2 Minimum Hardware Requirements 65

List of Figures

3.1 Feature Distribution Across Attack Families (Log1p Scale) 19
3.2 Class Distribution 20
3.3 Feature Correlation Matrix (sample n=15,000) 21
3.4 Packet Length Mean Density Distribution Across Attack Classes 22
3.5 Flow Duration Density Distribution Across Attack Classes 23
3.6 Flow Bytes/s Density Distribution Across Attack Classes 24
3.7 Forward Packet Length Mean Density Distribution Across Attack Classes . . . 25
3.8 Packet Length Mean Distribution by Attack Class 26
3.9 Flow Duration Distribution by Attack Class (Log Scale) 27
3.10 Flowchart of Random Forest 29
3.11 Flowchart of XGBoost 31
3.12 Project Pipeline 34

4.1 Feature Selection and Preprocessing 40
4.2 Scaling and Label Encoding 41
4.3 Data Partitioning and Class-Weight Adjustment Code 42
4.4 Base Classifier Implementation Code (Random Forest and XGBoost) 43
4.5 Ensemble Code 44
4.6 Top 20 Random Forest Feature Importances 45
4.7 Model Evaluation Code (Classification Report and Confusion Matrix) 47

5.1 Confusion Matrix - Ensemble (Test Set) 49
5.2 Per-Class Classification Metrics Heatmap - Ensemble 51

6.1 Detailed Flow Inspection and Class Probability Distribution 56
6.2 Real-Time Network Intrusion Monitoring Dashboard 57

7.1 Git Repository 58

Contents

Acknowledgement i

Abstract ii

List of Tables iii

List of Figures iv

1 Introduction 1

2 Supporting Literature 2

2.1 Literature Review . . . 2

2.2 Literature Review Summary . . . 8

2.3 Findings and Proposals . . . 9

3 System Analysis 10

3.1 Analysis of the Dataset . . . 10

3.1.1 About the Dataset . . . 10

3.1.2 Explore the Dataset . . . 11

3.2 Data Preprocessing . . . 12

3.2.1 Data Cleaning . . . 12

3.2.2 Analysis of Feature Variables . . . 12

3.2.3 Analysis of Class Variables . . . 18

3.3 Data Visualization . . . 20

3.4 Analysis of Architecture and Algorithms . . . 27

3.4.1 Detailed Study Report of the Proposed Algorithms . . . 28

3.5 Project Pipeline . . . 33

3.5.1 Project Pipeline Diagram and Development Plan . . . 33

3.6 Feasibility Analysis . . . 37

3.6.1 Technical Feasibility . . . 37

3.6.2 Economic Feasibility . . . 38

3.6.3 Operational Feasibility . . . 38

3.7 System Environment . . . 38

3.7.1 Software Environment . . . 38

3.7.2 Hardware Environment . . . 39

4 System Design **40**
4.1 Model Building . . . 40
4.1.1 Implementation Code . . . 40
4.1.2 Model Planning . . . 41
4.1.3 Training . . . 42
4.1.4 Model Evolution and Validation Across Development Milestones . . . 45
4.1.5 Testing . . . 47
5 Results and Discussion **48**
5.1 Comparison of Models . . . 48
5.2 Confusion Matrix . . . 49
5.3 Evaluation Metrics . . . 50
5.4 Metrics Calculated from the Confusion Matrix . . . 51
5.5 Discussion . . . 52
6 Model Deployment **53**
6.1 Model Implementation . . . 53
6.2 UI Design . . . 56
7 Git History **58**
8 Conclusions **59**
8.1 Implementation of the Real-Time IDS Pipeline . . . 59
8.2 Pipeline Integrity and Training Inference Parity . . . 59
8.3 Domain Generalization and Continuous Adaptation . . . 60
8.4 Critical Analysis of Operational Limitations . . . 60
8.5 Concluding Remarks . . . 61
9 Future Work **62**
9.1 Continual Learning and Adaptive Concept Drift Mitigation . . . 62
9.2 Explainable AI (XAI) and Real-Time SHAP Attributions . . . 62
9.3 Hardware Acceleration and Kernel-Level Packet Inspection . . . 62
9.4 Hybrid Unsupervised Zero-Day Threat Interception . . . 63
10 Appendix **64**
10.1 Minimum Software Requirements . . . 64
10.2 Minimum Hardware Requirements . . . 65
11 References **66**

1. Introduction

The rapid expansion of digital infrastructure has made computer networks a primary target for malicious actors. Global cybercrime damages are projected to exceed ten trillion dollars annually in 2025–2026, with India seeing sharp rises in DDoS and ransomware attacks targeting banking, healthcare, and government networks.

An Intrusion Detection System (IDS) monitors network traffic for unauthorized access or malicious behaviour. Signature-based systems match traffic against known attack patterns; anomaly-based systems flag deviations from a learned baseline. Signature-based approaches fail against zero-day exploits, driving research into ML-based detection capable of generalizing to unseen attacks.

A persistent gap exists between academic IDS research and practical deployment. Most published systems evaluate on static datasets without addressing real-time operational pipelines, continuous alerting, or analyst-facing interfaces, limiting operational value. A further challenge is severe class imbalance: benign flows dominate real traffic, and models that ignore this risk high false-negative rates on minority attack classes.

This project is motivated by CERT-In mandates for faster incident detection and the expanded network perimeter from hybrid cloud and remote work environments.

Objectives:

1. Train a Random Forest + XGBoost ensemble on the combined CIC-IDS2017 and CSE-CIC-IDS2018 benchmark for multi-class intrusion detection.
2. Implement a real-time alert mechanism upon detection of anomalous flows.
3. Build a real-time detection pipeline: live flow feature extraction $\rightarrow$ ensemble classification $\rightarrow$ alerting and telemetry.
4. Comparatively evaluate RF, XGBoost, and the ensemble on accuracy, precision, recall, and F1.

NIDAS is designed as an end-to-end real-time intrusion detection pipeline that processes network traffic via a custom CICFlowMeter implementation. Each flow is classified into: Normal Traffic, DoS, DDoS, Port Scanning, or Brute Force. Detected anomalies trigger timestamped alerts with predicted attack type and confidence score. A React JS dashboard surfaces live flow classifications; a FastAPI backend handles communication between the detection engine and UI.

The key contribution is end-to-end integration of ensemble detection with real-time alerting and dashboard visualization operationally actionable, not just accurate.

2. Supporting Literature

2.1 Literature Review

PAPER I

Ness S, Eswarakrishnan V, Sridharan H, Shinde V, Janapareddy N V P, & Dhanawat V. (2025). Anomaly detection in network traffic using advanced machine learning techniques. IEEE Access, vol. 13, pp. 16133–16149.

This paper investigates various machine learning models for detecting anomalies in network traffic, focusing on overcoming challenges like class imbalance and feature complexity. Accurate anomaly detection serves as a proactive measure to safeguard network integrity and identify dynamic security risks.

The authors utilize five machine learning algorithms: Isolation Forest, Naive Bayes, XGBoost, LightGBM, and Support Vector Machine (SVM). The study relies on the KDDCup'99 dataset, which includes normal activity and multiple attack types like DoS, Probe, Remote to Local (R2L), and User to Root (U2R). Data preprocessing involves the Synthetic Minority Over-sampling Technique (SMOTE) for balancing classes, label encoding for categorical features, and Principal Component Analysis (PCA) for feature extraction and dimensionality reduction.

The models were evaluated using 5-fold cross-validation with metrics including accuracy, precision, recall, and F1-score. LightGBM achieved the highest test accuracy (1.0) and F1-score (0.85). SVM also demonstrated strong performance with a test accuracy of 0.85 and F1-score of 0.86. Conversely, Isolation Forest performed poorly with a test accuracy of 0.40, struggling with the dataset's complexity.

The study concludes that advanced ensemble and gradient boosting methods like LightGBM and XGBoost, alongside SVM, offer robust detection capabilities for network traffic anomalies. Future work suggests integrating deep learning techniques like CNNs and RNNs, using ensemble methods to reduce false positives, and applying models to newer datasets like CICIDS and NSL-KDD.

Title	Anomaly Detection in Network Traffic Using Advanced Machine Learning Techniques
Area of Work	Evaluating and optimizing supervised and unsupervised ML models to detect anomalies in network traffic, addressing data imbalance and feature complexity.
Dataset	KDDCup'99 dataset containing normal traffic and multiple attack types (DoS, Probe, R2L, U2R). Balanced using SMOTE.
Methodology	Data preprocessing with missing value handling, label encoding, z-score normalization, and PCA. Five ML models trained with hyperparameter tuning and 5-fold cross-validation (80:20 split).
Algorithm	Isolation Forest, Naive Bayes, XGBoost, LightGBM, Support Vector Machine (SVM).
Result/Precision	LightGBM: Test Accuracy = 1.0, F1 = 0.85. SVM: Accuracy = 0.85, F1 = 0.86. XGBoost: Accuracy = 0.83, F1 = 0.84. Isolation Forest: Accuracy = 0.40.
Advantages	Comprehensive comparison of classical ML models. Highlights efficacy of gradient boosting and effectively uses SMOTE and PCA.
Future Proposal	Recommends deep learning (CNNs, RNNs), ensemble methods to reduce false positives, unsupervised approaches for zero-day attacks, and validation on CICIDS and NSL-KDD datasets.

Table 2.1: Summary of Paper 1

PAPER II

Fathima AHN, Ibrahim SPS, Khraisat A. Enhancing Network Traffic Anomaly Detection: Leveraging Temporal Correlation Index in a Hybrid Framework. IEEE Access. 2024 Sep 11;12:136805–136824.

This paper introduces the Temporal Metric-Driven GRU Embedded Generative Neural Network (TMG-GRU-VAE) to improve the identification of network traffic irregularities. Traditional models often emphasize detecting network attacks but struggle to capture the dynamic and temporal aspects of network traffic, making it difficult to detect subtle behavioral shifts.

The proposed methodology combines Gated Recurrent Units (GRU) with Variational Autoencoders (VAE) to effectively learn normal temporal patterns in sequence data. Additionally, the authors propose a novel Temporal Correlation Index (TCI) score to assess temporal behavior and establish dynamic anomaly detection thresholds. The study utilizes the CIC-IDS-2017 and CIC-IDS-2018 datasets, splitting the preprocessed data into 70% training and 30% testing subsets.

The models were evaluated using accuracy, precision, recall, F1-score, and false alarm rates. The TMG-GRU-VAE model combined with TCI achieved significant reductions in False Positives. Improvements in FP reduction ranged from 7.2% to 12.9% for CIC-IDS-2017, and 7.1% to 14.1% for CIC-IDS-2018. The GRU-VAE with TCI achieved the highest True Positive rates and the lowest FP rates.

The research demonstrates that integrating temporal correlation metrics with hybrid deep learning frameworks substantially reduces false alarms and improves overall anomaly detection accuracy. Future work aims to develop explainable AI models to enhance interpretability and confidence in detection results, particularly for addressing zero-day threats.

Title	Enhancing Network Traffic Anomaly Detection: Leveraging Temporal Correlation Index in a Hybrid Framework
Area of Work	Developing a hybrid deep learning framework (TMG-GRU-VAE) combined with a Temporal Correlation Index (TCI) to capture temporal dependencies and improve network anomaly detection.
Dataset	CIC-IDS-2017 and CIC-IDS-2018 datasets. Preprocessed with a 70:30 train-test ratio.
Methodology	Integrates GRUs into a VAE to learn sequence-based traffic behaviours. A novel TCI metric is computed by comparing original and reconstructed sequences to define dynamic anomaly thresholds.
Algorithm	Temporal Metric-Driven GRU Embedded Generative Neural Network (TMG-GRU-VAE), LSTM-VAE, Temporal Correlation Index (TCI) thresholding.
Result/Precision	GRU-VAE with TCI achieved the lowest FP rate (7.2% on CIC-IDS-2017, 7.1% on CIC-IDS-2018). Precision improved from 66.3% to 69.4% in specific configurations.
Advantages	Effectively manages feature complexities and temporal shifts. Drastically reduces false alarm rates while identifying subtle behavioral changes over time.
Future Proposal	Focuses on developing explainable AI models, and extending testing to larger-scale environments to assess scalability and zero-day attack resilience.

Table 2.2: Summary of Paper 2

PAPER III

Ji C, Liu H, Dai W. Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Two-Stage Feature Fusion. IEEE Access. 2025 Feb 4;13:27310–27324.

The paper proposes GRU-ResCBANet, a deep learning hybrid model tailored for network traffic anomaly detection. The model addresses issues such as low detection rates for minority attack classes and high false alarm rates by employing a parallel two-stage feature extraction and fusion technique.

In the first stage, the ResCBANet module uses an improved residual network enhanced with a Convolutional Block Attention Module (CBAM) to extract spatial features and channel attention. In parallel, a Gated Recurrent Unit (GRU) module captures temporal dependencies. These features are fused and passed to a fully connected classification layer using a Softmax activation function. The model was evaluated on the NSL-KDD, UNSW-NB15, and CICIDS2017 datasets, which were preprocessed and reduced in dimensionality using ANOVA for feature selection.

GRU-ResCBANet delivered state-of-the-art results. For binary classification, it achieved accuracies of 99.70% on NSL-KDD, 99.16% on UNSW-NB15, and 99.83% on CICIDS2017. For multi-class classification, accuracies reached 99.69%, 98.23%, and 99.79% respectively. The model consistently outperformed eight other comparison models, including SVM, RF, 2D-CNN, and CNN-LSTM.

The study concludes that parallel integration of GRU and ResCBANet successfully captures comprehensive feature representations, significantly boosting anomaly detection accuracy. Future directions include exploring model compression for edge computing, applying oversampling strategies for class imbalance, and integrating federated learning for privacy-preserving distributed security.

Title	Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Two-Stage Feature Fusion
Area of Work	Improving network traffic anomaly detection accuracy and reducing false alarm rates using a hybrid deep learning model that fuses spatiotemporal and channel features.
Dataset	NSL-KDD, UNSW-NB15, and CICIDS2017 datasets. Transformed into 3D structured inputs after dimensionality reduction via ANOVA.
Methodology	Parallel two-stage architecture. Stage one uses ResCBANet (CNN + Residual Units + CBAM) for spatial/channel features. Stage two uses GRUs for temporal features. Outputs fused and classified using a fully connected Softmax layer.
Algorithm	GRU-ResCBANet, Residual Convolutional Block Attention Network (ResCBANet), Gated Recurrent Units (GRU).
Result/Precision	Binary classification: 99.70% (NSL-KDD), 99.16% (UNSW-NB15), 99.83% (CICIDS2017). Multi-class: 99.69%, 98.23%, 99.79% respectively. Highest F1-scores among 8 baseline models.
Advantages	Parallel two-stage strategy efficiently leverages multidimensional traffic characteristics. Excels in detecting both frequent and complex attack types with high stability.
Future Proposal	Model pruning/quantization for edge devices, oversampling for UNSW-NB15 class imbalance, and federated learning for privacy-preserving distributed security.

Table 2.3: Summary of Paper 3

2.2 Literature Review Summary

Title	Summary
Ness S, et al. Anomaly Detection in Network Traffic Using Advanced Machine Learning Techniques. IEEE Access. 2025.	This study evaluates multiple ML models (Isolation Forest, Naïve Bayes, XGBoost, LightGBM, SVM) for network traffic anomaly detection using KDDCup'99. Using SMOTE and PCA, the authors demonstrated that LightGBM and SVM achieved the highest accuracy and F1-scores, confirming the effectiveness of ensemble and gradient boosting methods for robust network security.
Fathima AHN, et al. Enhancing Network Traffic Anomaly Detection: Leveraging Temporal Correlation Index in a Hybrid Framework. IEEE Access. 2024.	This paper introduces a hybrid deep learning framework (TMG-GRU-VAE) combining GRUs with VAEs to capture temporal dependencies in network traffic, alongside a novel TCI metric for dynamic detection thresholds. Evaluated on CIC-IDS-2017 and CIC-IDS-2018, the method drastically reduced false positive rates, proving that temporal correlation metrics significantly improve detection of subtle anomaly shifts.
Ji C, Liu H, Dai W. Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Two-Stage Feature Fusion. IEEE Access. 2025.	The authors propose GRU-ResCBANet, a hybrid model using parallel two-stage feature extraction. Stage one uses a CNN with Residual Units and CBAM for spatial/channel features; stage two uses GRUs for temporal features. Tested across NSL-KDD, UNSW-NB15, and CICIDS2017, the model achieved state-of-the-art accuracy in binary and multi-class classification.

Table 2.4: Literature Summary

2.3 Findings and Proposals

Findings

The reviewed literature establishes several consistent findings. Ensemble and fusion-based approaches uniformly outperform single-model classifiers in network traffic anomaly detection, across both traditional ML and deep learning paradigms. The CIC-IDS2017 dataset is the dominant benchmark in current IDS research and is the standard against which results from this project are comparable. Temporal features, particularly IAT-based features, carry significant discriminative information that improves detection of attack types with characteristic timing signatures. Despite the progress in detection accuracy, none of the three reviewed papers implement a real-time operational pipeline; all evaluate models in an offline, batch mode.

Identified Gap

The common gap across all three reviewed papers is the absence of a real-time alert mechanism. Each paper treats intrusion detection as a classification task on a static dataset, with no consideration of how detected anomalies are surfaced to an operator as they occur. This gap is practically significant: a model with high batch accuracy that cannot generate timely alerts provides no operational advantage over a post-hoc log analysis.

Proposal

This project proposes to close this gap by integrating a real-time alerting mechanism directly into the ensemble detection pipeline. The proposed system extends the soft-voting RF + XGBoost ensemble validated by Ness et al. with a FastAPI backend that classifies incoming flows in real time and triggers alert notifications upon detection of anomalous traffic. A React JS dashboard surfaces these alerts to the operator with predicted attack category and ensemble confidence score. This integration of detection and alerting constitutes the novel contribution of this project over the reviewed literature.

3. System Analysis

3.1 Analysis of the Dataset

3.1.1 About the Dataset

To establish a resilient intrusion detection system capable of mitigating domain-specific vulnerabilities, this project integrates two widely recognized benchmark datasets developed by the Canadian Institute for Cybersecurity: CIC-IDS2017 and CSE-CIC-IDS2018. Both datasets simulate realistic enterprise network traffic containing diverse, state-of-the-art attack executions alongside normal background user activity, captured using the network traffic profiling tool CICFlowMeter.

The CIC-IDS2017 benchmark comprises eight raw CSV captures totaling 2,830,743 network flow records across 79 columns. The CSE-CIC-IDS2018 benchmark, collected in a collaborative effort between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC) on AWS infrastructure, encompasses six targeted attack captures used in this study (BruteForce, BruteForce2, BruteForceSSH, DDoS, DoS, and DoS2) totaling 6,291,449 raw flow records across 80 columns. To harmonize the two heterogeneous schemas, abbreviated column headers in the 2018 dataset (e.g., Dst Port, Tot Fwd Pkts, Pkt Len Mean) were systematically mapped to the standardized naming conventions of CIC-IDS2017. Internal repeated header lines and non-flow metadata attributes (Protocol and Timestamp) were excluded, yielding a combined pre-cleaning corpus of 9,117,071 flow records across 77 shared numeric features.

The raw multiclass labels across both datasets were consolidated into five harmonized traffic categories:

- **Normal Traffic:** Merging BENIGN from CIC-IDS2017 and benign from CSE-CIC-IDS2018.
- **Denial of Service (DoS):** Aggregating DoS Hulk, DoS GoldenEye, DoS slowloris, and DoS Slowhttptest from 2017 with dos attacks-goldeneye, dos attacks-slowloris, dos attacks-hulk, and dos attacks-slowhttptest from 2018.
- **Distributed Denial of Service (DDoS):** Combining DDoS from 2017 with ddos attacks-loic-http, ddos attack-loic-udp, and ddos attack-hoic from 2018.
- **Brute Force:** Unifying credential-guessing attacks including FTP-Patator and SSH-Patator from 2017 with ftp-bruteforce and ssh-bruteforce from 2018.
- **Port Scan:** Incorporating port scanning probe activity (PortScan from 2017).

Peripheral attack categories outside these five focus domains (such as Bot, Infiltration, Web Attacks, and Heartbleed) were excluded, resulting in a cohesive multi-class dataset tailored for tabular machine learning algorithms.

3.1.2 Explore the Dataset

The combined dataset initially encompasses 9,117,071 flow records across 77 numeric features and two tracking attributes (Attack Type and Source). Statistical profiling of the combined corpus revealed several critical data hygiene and distribution challenges:

- **Constant Features:** Eight feature columns exhibited zero variance across all 9.1 million records: Bwd PSH Flags, Bwd URG Flags, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, and Bwd Avg Bulk Rate. Carrying no discriminative signal, these columns were slated for immediate removal, reducing the feature space to 69 numerical dimensions.
- **Missing and Infinite Values:** A total of 26,026 records contained infinite or NaN values in rate-dependent features (Flow Bytes/s and Flow Packets/s), caused by zero-duration flows dividing by zero time increments. These corrupted records represent under 0.29% of the raw corpus and were removed.
- **Duplicate Records:** An extensive deduplication pass identified 2,263,887 duplicate flow records (approximately 24.9% of valid rows). Removing these duplicates reduced the corpus to 6,827,158 unique records, preventing artificial inflation of model metrics and data leakage between train and test splits.
- **Severe Class Imbalance:** Among the 6,827,158 unique flows, benign background traffic heavily predominated with 5,914,415 records (86.63%), compared to 390,200 DoS records (5.72%), 328,605 DDoS records (4.81%), 103,244 Brute Force records (1.51%), and 90,694 Port Scan records (1.33%).

Descriptive statistics across the unified flow metrics highlight stark distributional skews. Flow Duration spans from 0 to 120 million microseconds with a low median, reflecting that the vast majority of legitimate and probe flows are short-lived, whereas DoS and slowloris attacks sustain extended connection durations. Packet length metrics exhibit strong bimodal tendencies: probe and scanning vectors concentrate near minimal payload thresholds (e.g., 3 to 6 bytes), whereas high-volume DDoS flooding attacks generate large payloads (mean exceeding 700 bytes).

3.2 Data Preprocessing

To transform the 9.12 million raw flow observations into an optimal, high-fidelity training dataset, a systematic data preprocessing and refinement pipeline was implemented.

3.2.1 Data Cleaning

A systematic five-stage data cleaning and preparation pipeline was applied to ensure dataset integrity and remove non-informative features:

1. **Zero-Variance Column Removal:** The 8 constant columns identified during exploration were dropped, retaining 69 informative numeric features.
2. **Infinity and Null Value Cleaning:** All infinite values ($\pm\infty$) were converted to NaN, and the 26,026 affected rows were eliminated, leaving 9,091,045 clean records.
3. **Deduplication:** Exact duplicate rows across the feature space were purged, eliminating 2,263,887 records and yielding 6,827,158 unique flow records.
4. **Benign Subsampling and Class Balancing:** In the unique corpus, Normal Traffic accounted for over 5.91 million records (86.6%). Retaining all benign records would induce severe majority-class bias and excessive training overhead without adding behavioral variance. Consequently, Normal Traffic was randomly subsampled at 15% (887,162 records) and combined with 100% of all attack records (912,743 flows). This produced a balanced, highly representative dataset of 1,799,905 records composed of:
   - Normal Traffic: 887,162 records (49.29%)
   - DoS: 390,200 records (21.68%)
   - DDoS: 328,605 records (18.26%)
   - Brute Force: 103,244 records (5.74%)
   - Port Scan: 90,694 records (5.04%)

   In terms of dataset origin, CSE-CIC-IDS2018 contributed 1,064,113 flows (59.12%) and CIC-IDS2017 contributed 735,792 flows (40.88%), ensuring robust cross-environment representation.
5. **Categorical Label Encoding:** The target categorical variable (Attack Type) was mapped to integers using scikit-learn's LabelEncoder, assigning integer indices 0 to 4 corresponding to Brute Force, DDoS, DoS, Normal Traffic, and Port Scan.

3.2.2 Analysis of Feature Variables

The unified dataset contains 69 candidate numeric features after removing zero-variance columns. Following the cleaning, feature selection, and domain-refinement pipeline, 40 features are

retained for model training and deployment. These can be grouped into the following categories based on the aspect of network flow behaviour they capture.

Packet length features capture the size characteristics of packets in both forward (client-to-server) and backward (server-to-client) directions, including maximum, minimum, mean, and standard deviation values. Feature importance analysis identifies this group as the strongest family of discriminative features, with Bwd Packet Length Mean, Packet Length Std, Bwd Packet Length Std, and Fwd Packet Length Mean exhibiting high discriminative capacity. These features are particularly discriminative for DoS and DDoS attacks, which generate packets of characteristic sizes at high rates.

Inter-Arrival Time (IAT) features measure the time intervals between successive packets within a flow, computed separately for the overall flow, the forward sub-flow, and the backward sub-flow. Standard deviation, maximum, and minimum IAT values are included. As demonstrated by Fathima et al. (2024), temporal features of this nature carry significant discriminative information for attacks that exhibit periodic or bursty timing patterns.

Traffic volume features include Total Length of Fwd Packets, Total Fwd Packets, and Total Backward Packets, characterizing the overall data volume transferred in each direction. These complement packet length features for detecting volume-based attacks.

Flow-level throughput features include Flow Bytes/s, Flow Packets/s, and Bwd Packets/s, which characterize the volumetric rate of the flow.

TCP flag features record the presence of FIN, PSH, and ACK flags within the flow. These help distinguish connection setup and teardown patterns associated with specific attack types such as port scans.

TCP window and header features include the initial TCP window sizes in both directions, forward and backward header lengths, and minimum segment size, which reflect protocol-level behaviour that varies across attack types.

Active and Idle time features capture the periods of activity and inactivity within a flow. Active Max and Idle Mean are retained after feature selection.

Feature selection and refinement were executed in four stages:

1. **Collinearity Filtering ($r > 0.98$):** Pearson correlation analysis evaluated on a stratified 50,000-sample subset revealed strong linear dependencies among subflow counters, bidirectional segment statistics, and forward/backward duplicates. Sixteen redundant collinear features were pruned: Fwd IAT Total, Subflow Fwd Packets, Subflow Bwd Packets, Subflow Fwd Bytes, Subflow Bwd Bytes, Avg Fwd Segment Size, Avg Bwd Segment Size, Fwd PSH Flags, Fwd URG Flags, ECE Flag Count, Average Packet Size,

Fwd IAT Max, Fwd Packets/s, Idle Max, Idle Min, and Down/Up Ratio. This reduced the feature space from 69 to 53 features.

2. **Kruskal-Wallis Hypothesis Testing:** A non-parametric Kruskal-Wallis H-test evaluated whether each feature exhibited statistically significant distribution differences across the five target classes. CWE Flag Count failed to demonstrate significance ($p = 0.612 > 0.05$) and was removed, reducing the feature space to 52 features.
3. **Random Forest Feature Importance Pruning:** A Random Forest classifier was fitted on the sample to compute Gini impurity-based feature importances. Features falling within the bottom 10% quantile were eliminated: URG Flag Count, Active Mean, Active Min, Idle Std, Active Std, and SYN Flag Count, yielding 46 candidate features.
4. **Domain Generalization and Redundancy Pruning:** Prior to model training, six additional features were eliminated: Destination Port (pruned to prevent shortcut learning and overfitting to static port numbers, ensuring port-agnostic generalization), Packet Length Variance (exact mathematical collinearity with Packet Length Std), Total Length of Bwd Packets, Bwd Packet Length Min, Fwd IAT Mean, and Bwd IAT Mean (which showed high multi-collinearity with overall flow timing and packet statistics).

This rigorous pruning process established the final, high-impact set of 40 features used for model training, offline evaluation, and live inference. Feature scaling using RobustScaler was fitted exclusively on the training split to standardize features with wide dynamic ranges and heavy-tailed distributions while providing resilience against extreme network burst outliers, followed by discriminative feature weighting.

The complete dictionary of the 40 retained bidirectional flow features is systematically organized into four functional groups in Table 3.1, Table 3.2, Table 3.3, and Table 3.4. Each entry details the operational metric name, analytical category, and behavioral security significance for distinguishing legitimate traffic from malicious exploits.

Feature Name	Category	Definition and Security Significance
Total Fwd Packets	Traffic Volume	Total count of packets sent from client to server. Abnormally high in volumetric upload floods.
Total Backward Packets	Traffic Volume	Total count of packets sent from server to client. Identifies asymmetric flows with missing responses.
Total Length of Fwd Packets	Traffic Volume	Cumulative payload bytes in forward packets. Differentiates heavy uploads from small control packets.
Fwd Packet Length Max	Packet Length	Maximum forward packet size in bytes. Captures maximum transmission unit (MTU) boundaries.
Fwd Packet Length Min	Packet Length	Minimum forward packet size in bytes. Detects tiny probe or TCP handshake packets.
Fwd Packet Length Mean	Packet Length	Arithmetic mean of forward packet sizes. Highly discriminative between scans and content streams.
Fwd Packet Length Std	Packet Length	Standard deviation of forward packet lengths. Measures payload variability across the transmission.
Bwd Packet Length Max	Packet Length	Maximum backward packet size in bytes. Distinguishes rich server responses from bare RST/ACKs.
Bwd Packet Length Mean	Packet Length	Arithmetic mean of backward packet sizes. Elevated in authentic web downloads, zero in unacknowledged floods.
Bwd Packet Length Std	Packet Length	Standard deviation of backward packet sizes. Measures response payload variability under load.

Table 3.1: Flow Volume and Packet Length Statistics (Features 1–10)

Feature Name	**Category**	**Definition and Security Significance**
Flow Duration	Temporal Dynamics	Total duration of bidirectional flow in μs. Separates persistent DoS floods from fast scans.
Flow Bytes/s	Throughput Rate	Total flow byte volume divided by flow duration. Spikes sharply during aggressive bandwidth floods.
Flow Packets/s	Throughput Rate	Total packet count divided by flow duration. Key metric for packet-per-second (pps) flooding detection.
Flow IAT Mean	Inter-Arrival Time	Mean interval between successive packets in flow (μs). Quantifies overall packet transmission pacing.
Flow IAT Std	Inter-Arrival Time	Standard deviation of inter-arrival times. Separates automated flood scripting from natural human jitter.
Flow IAT Max	Inter-Arrival Time	Maximum elapsed interval between any two packets in flow. Identifies idle gaps in active sessions.
Flow IAT Min	Inter-Arrival Time	Minimum interval between successive packets. Detects back-to-back burst packet transmission.
Fwd IAT Min	Inter-Arrival Time	Minimum interval between consecutive client packets. Diagnostic of rapid-fire attack generators.
Fwd IAT Std	Inter-Arrival Time	Standard deviation of forward inter-arrival times. Reflects jitter consistency in client-side transmission.
Bwd IAT Total	Inter-Arrival Time	Cumulative inter-arrival time across backward packets. Quantifies server-side interaction duration.

Table 3.2: Flow Throughput and Inter-Arrival Timing Dynamics (Features 11–20)

Feature Name	Category	Definition and Security Significance
Bwd IAT Std	Inter-Arrival Time	Standard deviation of backward packet intervals. Reflects server response jitter and processing latency.
Bwd IAT Max	Inter-Arrival Time	Maximum interval between backward packets. Detects long server processing delays or stalled sessions.
Bwd IAT Min	Inter-Arrival Time	Minimum interval between backward packets. Detects automated server reflection or response bursts.
Fwd Header Length	Protocol Header	Total octets in forward transport headers. Measures encapsulation overhead across client packets.
Bwd Header Length	Protocol Header	Total octets in backward transport headers. Separates TCP headers from UDP datagram headers.
Bwd Packets/s	Throughput Rate	Rate of backward packet arrivals: $N_{bwd}/\text{Duration}$. Abnormally low in unidirectional flood attacks.
Min Packet Length	Packet Length	Minimum packet size across both flow directions. Detects zero-payload ACK or SYN sweeps.
Max Packet Length	Packet Length	Maximum packet size across both flow directions. Captures standard Ethernet MTU ceiling (≈ 1500 bytes).
Packet Length Mean	Packet Length	Global mean packet size: Total Bytes/Total Packets. Primary indicator of overall flow volumetric profile.
Packet Length Std	Packet Length	Global standard deviation of packet sizes. Distinguishes fixed-size attack packets from variable traffic.

Table 3.3: Directional Timing and Protocol Header Attributes (Features 21–30)

Feature Name	Category	Definition and Security Significance
FIN Flag Count	TCP Control Bit	Count of packets with FIN asserted. Differentiates graceful termination from sudden connection aborts.
RST Flag Count	TCP Control Bit	Count of packets with RST asserted. Frequently elevated during port scans targeting closed ports.
PSH Flag Count	TCP Control Bit	Count of packets with PSH asserted. Indicates urgent payload push in active application transactions.
ACK Flag Count	TCP Control Bit	Count of packets with ACK asserted. Essential for verifying three-way handshake completion.
Init_Win_bytes_forward	TCP Windowing	Advertised TCP window in initial forward packet. Characteristic of client OS network stack configuration.
Init_Win_bytes_backward	TCP Windowing	Advertised TCP window in initial backward packet. Reflects server OS network stack configuration.
act_data_pkt_fwd	Payload Activity	Count of forward packets with ≥ 1 byte of payload. Zero in pure TCP synchronization probes.
min_seg_size_forward	Protocol Header	Minimum segment size observed in forward packets. Encodes client-advertised TCP MSS options.
Active Max	Flow State	Maximum active duration before idle interval (μs). Identifies sustained active transmission bursts.
Idle Mean	Flow State	Mean duration of inactivity intervals between packet bursts. Distinguishes continuous from periodic traffic.

Table 3.4: TCP Control Flags, Windowing, and State Dynamics (Features 31–40)

3.2.3 Analysis of Class Variables

To understand how effectively different network features distinguish between benign traffic and various attack families, a class-separation analysis was conducted across the 1,799,905 records spanning the five traffic classes.

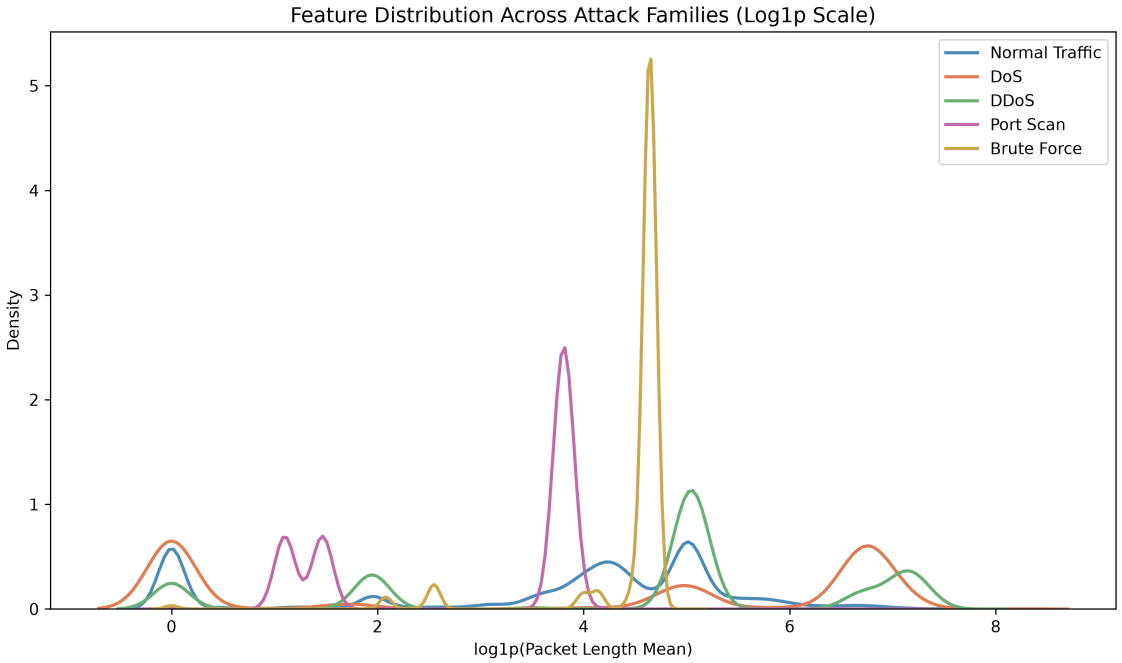


Figure 3.1: Feature Distribution Across Attack Families (Log1p Scale)

Visualizing the features on a logarithmic scale, as shown in Figure 3.1, reveals distinct behavioral patterns across different attack categories. Packet Length Mean shows strong class separation: Port Scan traffic produces an extremely sharp, narrow spike at very low values, consistent with its near-zero mean packet length of 3.25 bytes. DDoS traffic concentrates at a high value with a tight distribution, reflecting its mean of 736.9 bytes. DoS is similarly elevated at 565.7 bytes but more spread out. Normal Traffic and Brute Force overlap considerably in the mid-range.

Flow Duration shows the clearest separation for Port Scan, which produces a sharp isolated spike at very low durations (mean 82,820 μs), consistent with rapid probe-style connections. DoS flows have the longest durations (mean 5.57×10^7 μs), forming a distinct cluster at the high end.

While certain attack classes exhibit highly specific, isolated signatures in individual features, substantial overlap exists between Normal Traffic and attack classes across the lower ranges of most features. No single feature provides a clean decision boundary across all five classes. This analysis supports the use of a multi-feature, non-linear ensemble approach capable of exploiting the combined discriminative signal across all 40 selected features.

3.3 Data Visualization

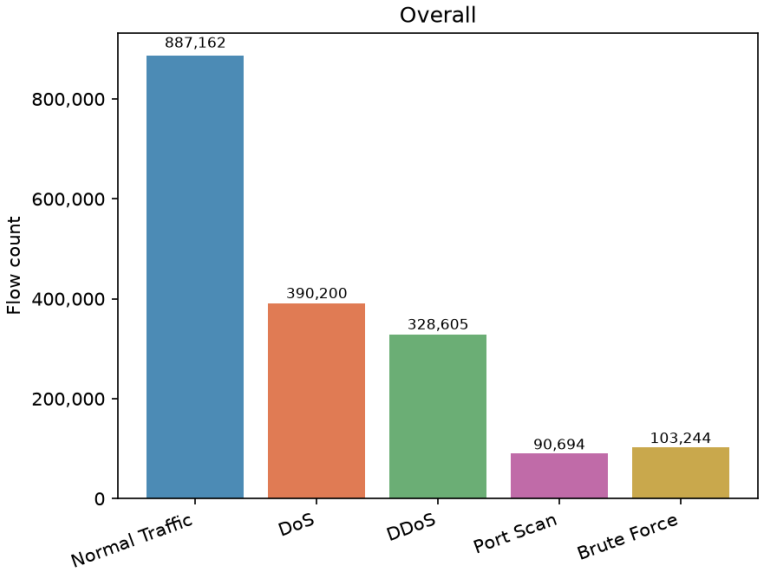


Figure 3.2: Class Distribution

Figure 3.2 illustrates the class distribution of the curated 1,799,905-flow benchmark. Following the 15% subsampling of benign records, Normal Traffic represents 49.3% of the corpus (887,162 flows), while DoS (21.7%, 390,200 flows) and DDoS (18.3%, 328,605 flows) constitute the primary attack vectors. The minority attack classes, Brute Force (5.7%, 103,244 flows) and Port Scan (5.0%, 90,694 flows), maintain substantial absolute sample counts sufficient to train deep decision boundaries without severe minority underfitting.

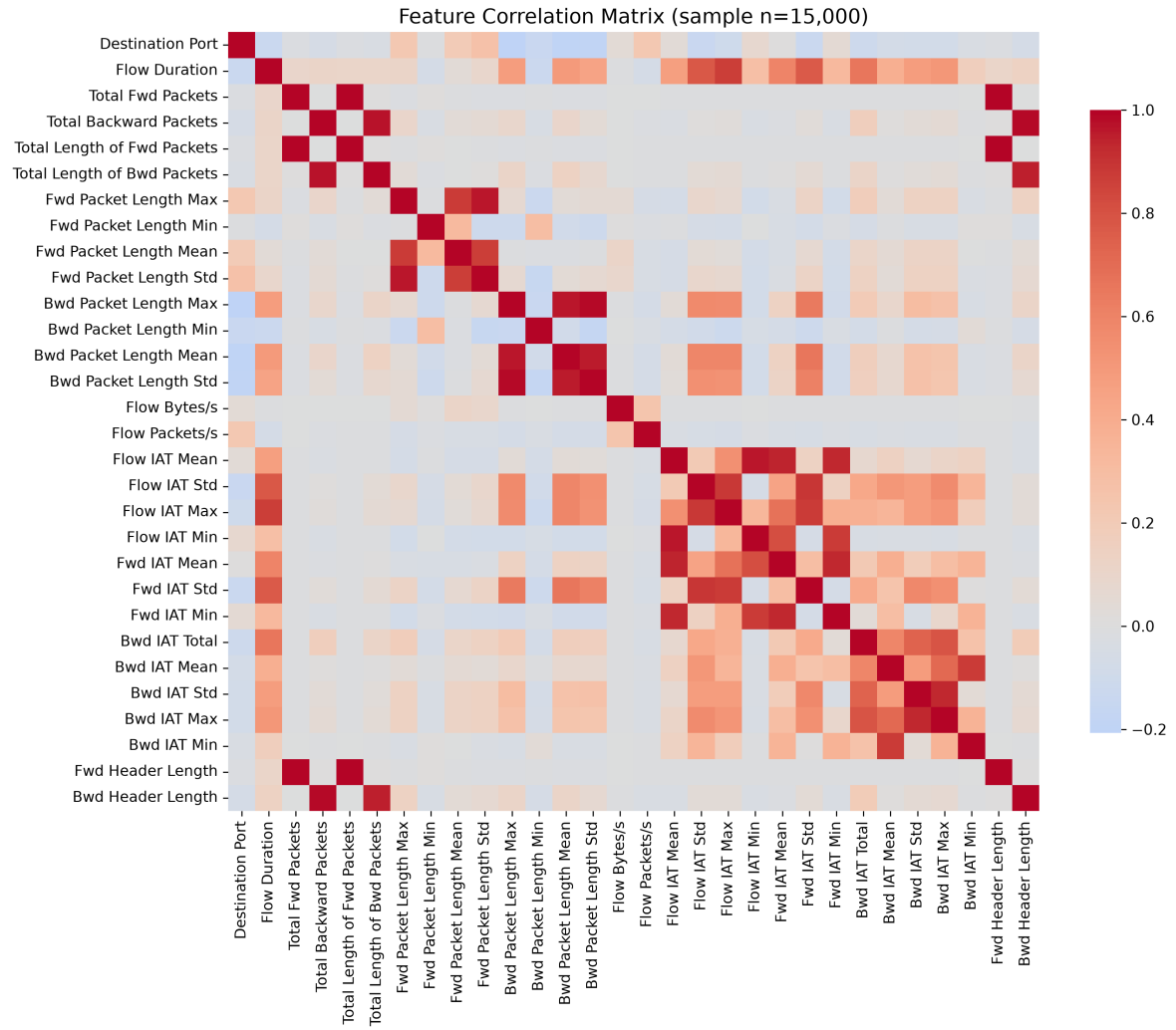


Figure 3.3: Feature Correlation Matrix (sample n=15,000)

Figure 3.3 displays the Pearson correlation matrix computed on a random sample of 15,000 records across the full feature set. The heatmap reveals several distinct clusters of high redundancy, with 27 feature pairs exceeding a correlation threshold of 0.99. The most striking redundancies are perfect correlations ($r = 1.0000$): Total Fwd Packets with Subflow Fwd Packets, Total Backward Packets with Subflow Bwd Packets, Total Length of Fwd Packets with Subflow Fwd Bytes, Fwd Packet Length Mean with Avg Fwd Segment Size, Bwd Packet Length Mean with Avg Bwd Segment Size, and Fwd Header Length with Fwd Header Length.1. These findings directly informed the correlation-based feature reduction step in the preprocessing pipeline, reducing the feature set from 69 to 53 features prior to statistical, importance-based, and domain-refinement selection.

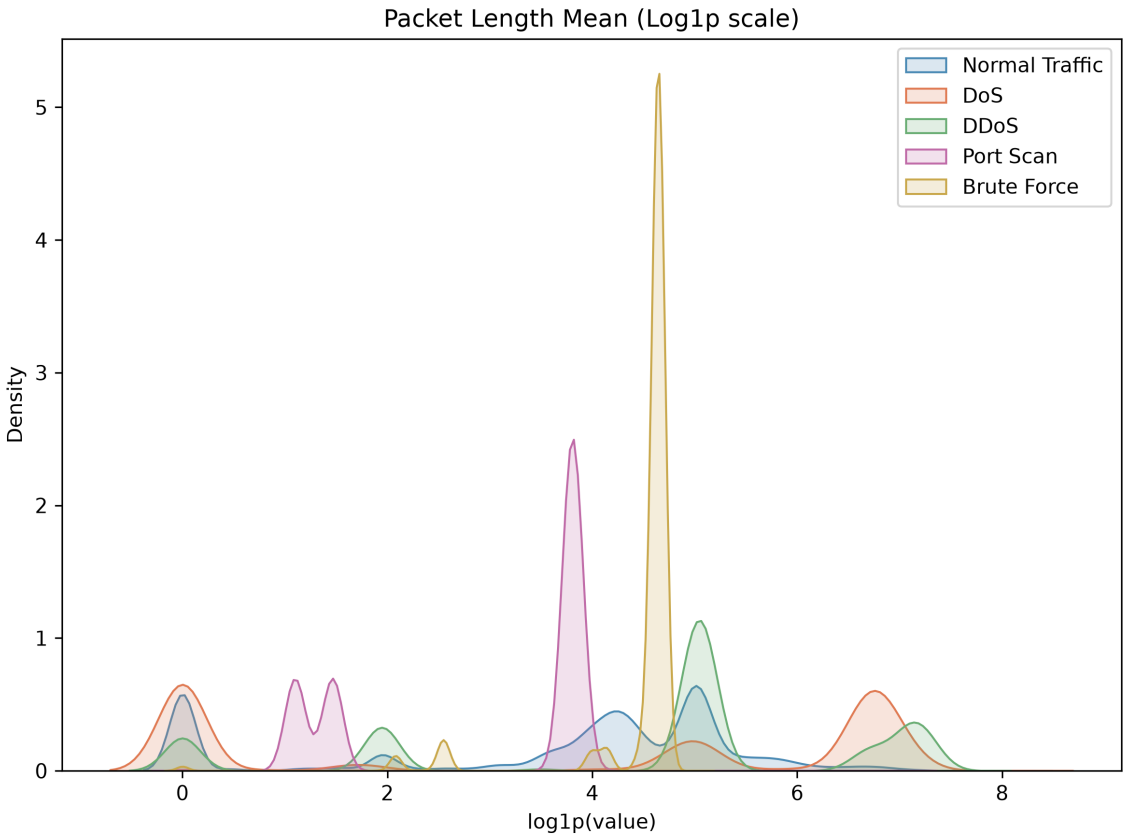


Figure 3.4: Packet Length Mean Density Distribution Across Attack Classes

Figure 3.4 presents the empirical density distribution of Packet Length Mean across the five traffic categories on a log1p scale. The distribution is heavily right-skewed, with normal traffic concentrating in the lower-to-mid range. Port Scan traffic produces an extremely sharp, isolated spike at near-zero values, reflecting rapid probe packets carrying minimal payloads. In contrast, DoS and DDoS flows cluster at substantially higher packet length values with tightly bounded distributions, characteristic of sustained packet flooding with consistent payload sizing.

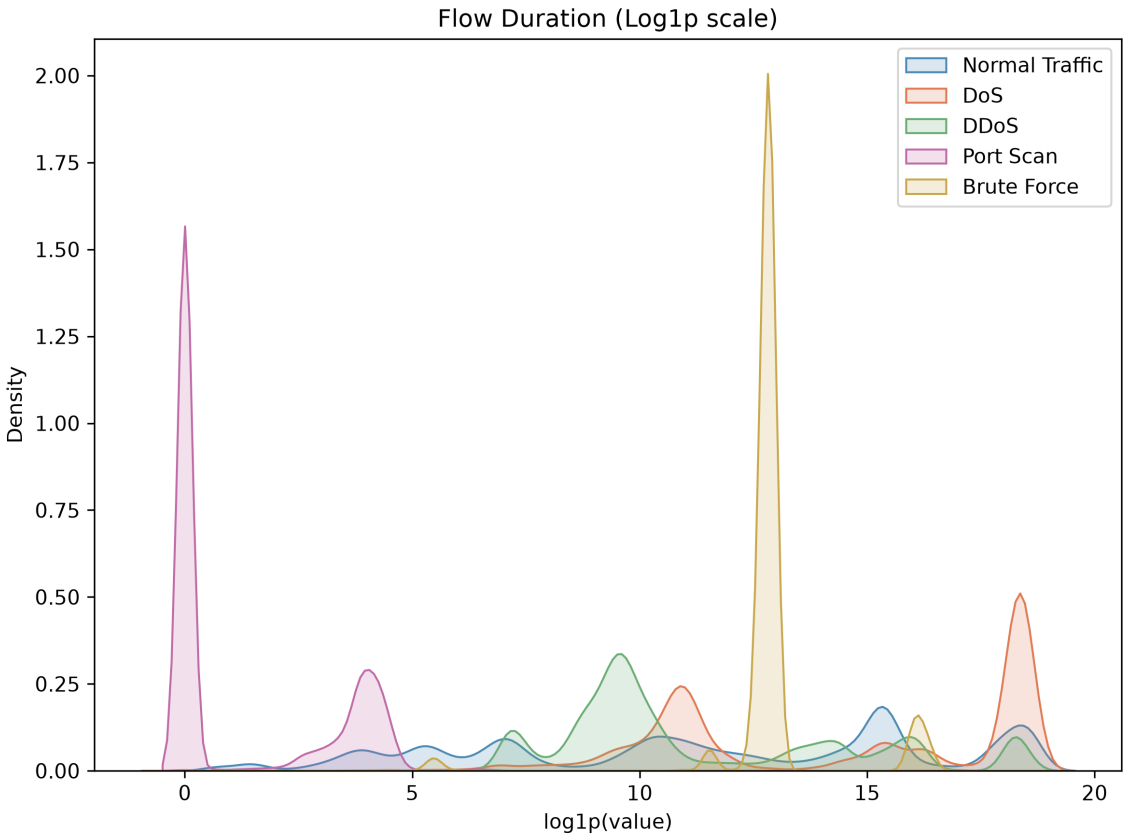


Figure 3.5: Flow Duration Density Distribution Across Attack Classes

Figure 3.5 displays the density distribution of Flow Duration on a log1p scale across each attack category. Port Scan activity exhibits a steep density concentration at very short durations, corresponding to rapid, transient connection attempts across target port ranges. Conversely, DoS traffic spans the highest duration spectrum, driven by long-lived sessions designed to exhaust server connection tables and socket buffers. Normal traffic and Brute Force exhibit intermediate duration profiles, providing strong discriminatory boundaries.

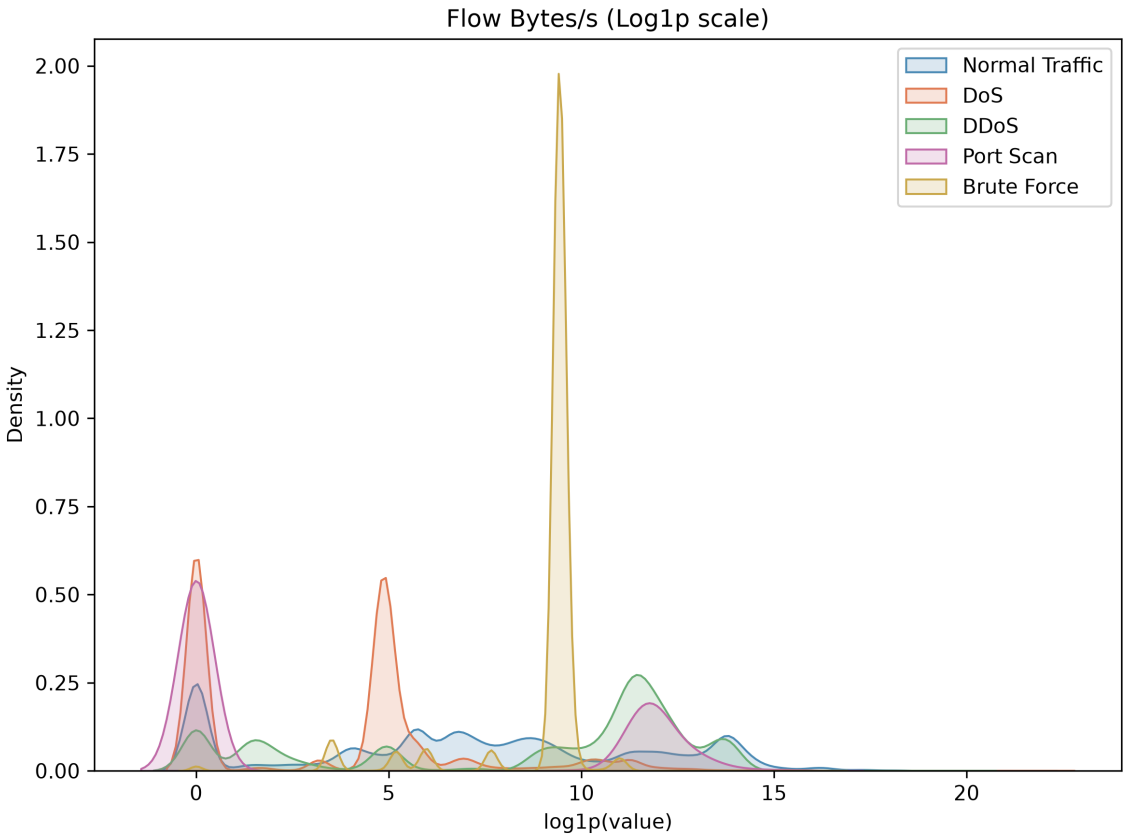


Figure 3.6: Flow Bytes/s Density Distribution Across Attack Classes

Figure 3.6 illustrates the distribution of flow throughput measured in Flow Bytes per second. Normal traffic spreads broadly across lower throughput tiers, reflecting varied application workloads. Port Scan manifests as a distinctive high-rate peak due to rapid transmission of minimal probe frames within brief microsecond windows. DoS traffic concentrates in a prominent mid-to-high throughput band, distinguishing volumetric flooding from standard interactive communications.

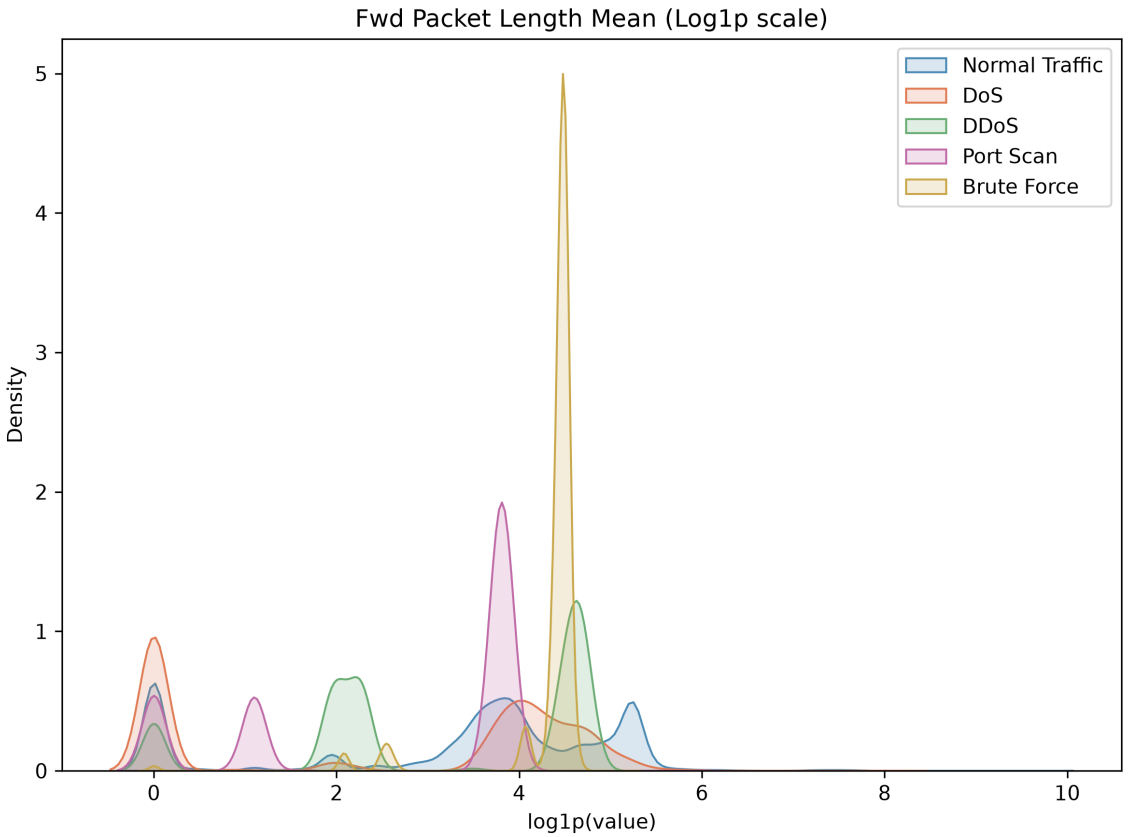


Figure 3.7: Forward Packet Length Mean Density Distribution Across Attack Classes

Figure 3.7 shows the density distribution of Forward Packet Length Mean across the traffic categories. This feature demonstrates pronounced class separability: Brute Force and DDoS generate extremely narrow, high-density spikes at small forward packet lengths, clearly distinguishable from the broader profile of Normal Traffic. The visible separation between classes across Figures 3.4–3.7 confirms the relevance of these individual metrics as discriminative inputs to the ensemble classifier.

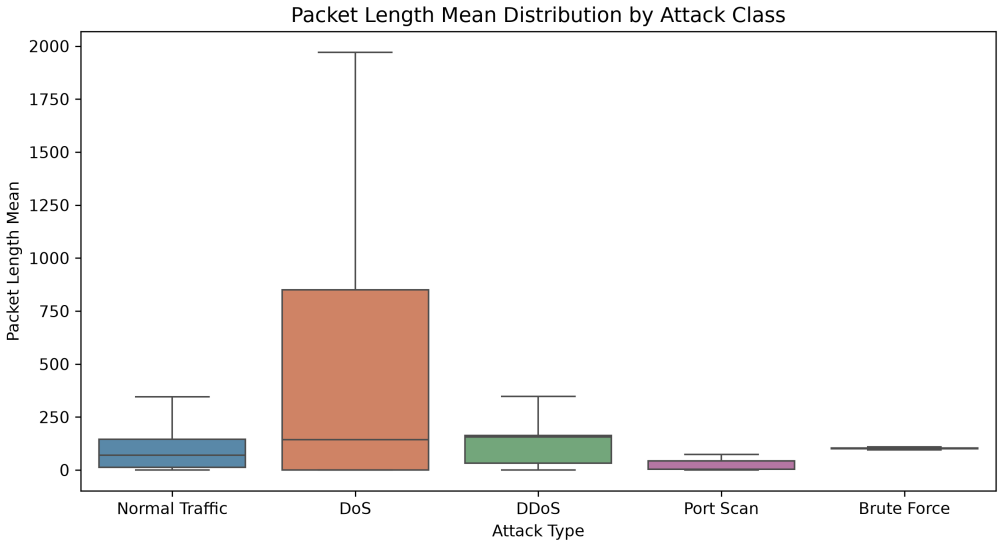


Figure 3.8: Packet Length Mean Distribution by Attack Class

Figure 3.8 presents box plots of Packet Length Mean across the five traffic classes. DDoS and DoS show substantially higher median packet lengths, approximately 800 bytes each, with DDoS exhibiting a wider interquartile range indicating greater variability in payload size. Normal Traffic has a low median of approximately 80 bytes with a moderate spread. Port Scan and Brute Force both have medians near zero, consistent with the short probe packets characteristic of scanning activity and the small credential packets used in brute force attempts. These inter-class differences in packet length make it a strongly discriminating feature for the ensemble model, as confirmed by its top-10 ranking in Random Forest feature importance.

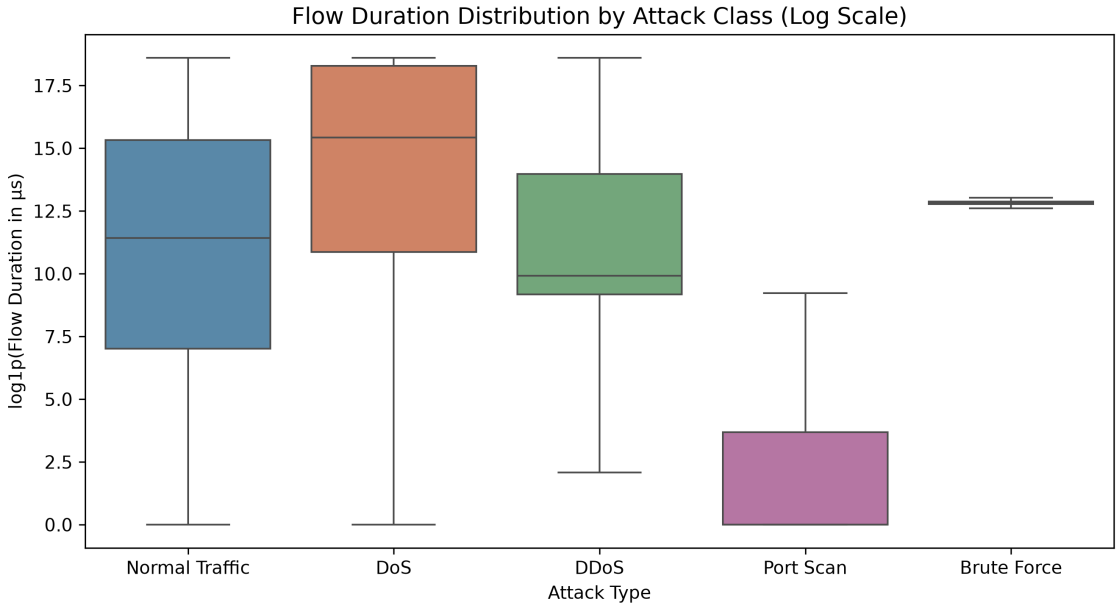


Figure 3.9: Flow Duration Distribution by Attack Class (Log Scale)

Figure 3.9 shows box plots of Flow Duration on a log1p scale across the five traffic classes. DoS flows exhibit by far the longest durations, consistent with sustained flooding behaviour that keeps connections open for extended periods. DDoS, Brute Force, and Normal Traffic all have low medians compressed near zero relative to DoS, though with varying spreads. Port Scan shows the shortest and most tightly concentrated durations, corresponding to its rapid, probe-style connection pattern. Flow Duration thus provides strong discriminative signal, particularly for separating DoS from all other classes.

3.4 Analysis of Architecture and Algorithms

The Real-Time Network Intrusion Detection and Alert System (NIDAS) employs a hybrid ensemble learning architecture designed to deliver high-throughput, low-latency threat detection across live network traffic. Structured network telemetry is characterized by extreme non-linearities, heavy-tailed burst distributions, and severe class imbalances between benign and malicious flows. Single-model classifiers frequently suffer from an operational trade-off: high-variance models overfit to benign network fluctuations, generating excessive false alarms, whereas high-bias models fail to capture stealthy, low-volume intrusions. To overcome these limitations, NIDAS couples Random Forest and Extreme Gradient Boosting (XGBoost) within a soft-voting ensemble framework, combining the variance-reduction strengths of bagging with the bias-reduction precision of gradient boosting. The following sections provide an in-depth analysis of the mathematical foundations, operational strengths, inherent weaknesses, and

project-specific scope of each proposed algorithm.

3.4.1 Detailed Study Report of the Proposed Algorithms

Random Forest

Random Forest is a non-parametric ensemble learning method based on bootstrap aggregating (bagging) of unpruned or depth-constrained decision trees. During training, the algorithm constructs $B = 200$ individual classification trees using bootstrap samples drawn with replacement from the 1.58 million training flows. At each node split, instead of evaluating all candidate dimensions across the feature space, a random subset of size $m = 0.4 \times 40 = 16$ features is sampled. Splitting criteria evaluate impurity reduction (such as Gini impurity, defined as $\text{Gini}(p) = 1 - \sum_{c=1}^{C} p_c^2$) to recursively partition the high-dimensional flow metric space. The final prediction for an input flow vector x is formed by aggregating posterior probability distributions across all individual trees:

$$P_{\text{RF}}(y = c \mid x) = \frac{1}{B} \sum_{b=1}^{B} P_b(y = c \mid x) \tag{3.1}$$

This dual randomization protocol (subsampling training instances and subsampling candidate features) effectively decorrelates individual decision trees, substantially lowering the overall variance of the ensemble without elevating its bias. Restricting the maximum tree depth to 10 ($max_depth = 10$) prevents individual estimators from overfitting to transient network artifacts or memorizing specific port signatures.

Strengths: Exceptional resistance to overfitting on high-dimensional tabular data through variance dampening; native resilience to noisy, skewed, or collinear network traffic features without requiring strict Gaussian assumptions; invariance to monotonic feature transformations, ensuring stable decision thresholds across metrics spanning several orders of magnitude; transparent model interpretability via Mean Decrease in Impurity (MDI) feature importance rankings; and high computational parallelization during training across all available CPU cores ($n_jobs = -1$).

Weaknesses: Significant memory footprint during deployment because the complete structure of 200 multi-class decision trees must reside concurrently in RAM; inference latency scales directly with tree count and depth, presenting potential computational overhead during extreme packet floods compared to compact linear models; inability to extrapolate beyond the numerical boundaries observed during training; tendency to favor majority traffic classes if class imbalances are unweighted; and redundant tree node partitions when processing correlated flow characteristics.

Scope of the algorithm with respect to this project: Within the NIDAS architecture, Random Forest provides the stable, variance-reducing foundation necessary to handle real-world network burstiness. Real-time packet telemetry frequently exhibits severe outliers and multi-modal distributions that cause volatile predictions in single-tree or parametric detectors. Operating on the 40 preprocessed flow features with balanced sample weighting, Random Forest reliably anchors classification boundaries across large volumes of normal traffic while preserving sensitivity to minority attack patterns. In deployment, its tree traversal executes in under 0.6 milliseconds per flow, ensuring continuous operation well within the sub-millisecond latency budget required for live socket monitoring.

Justification: Ness et al. (2025), in a multi-model comparison on network traffic anomaly detection, demonstrated that tree-based ensemble methods consistently outperform single-model classifiers on structured network flow data. Fathima et al. (2024) validate their hybrid detection framework on the CIC-IDS2017 dataset, the same benchmark used here, directly supporting Random Forest's inclusion as a base learner.

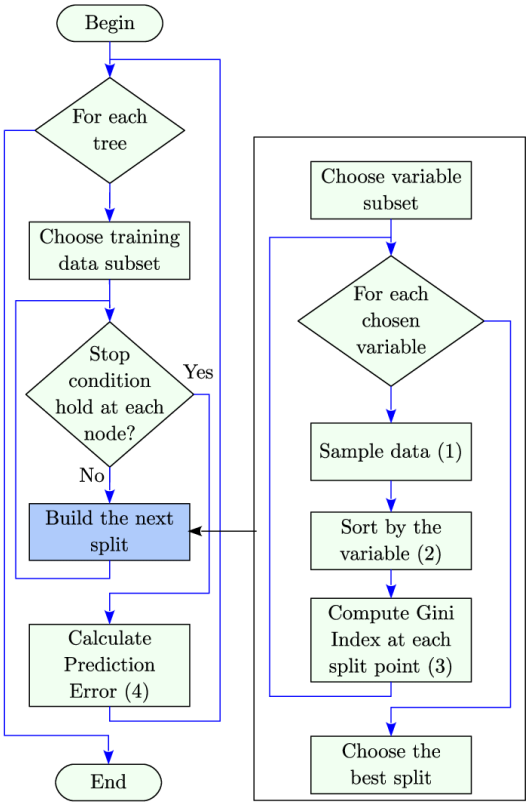


Figure 3.10: Flowchart of Random Forest

XGBoost

Extreme Gradient Boosting (XGBoost) is a scalable and highly optimized gradient tree boosting framework that constructs an ensemble of decision trees sequentially. Rather than building trees independently in parallel, XGBoost builds trees additively: at each iteration t, a new regression tree $f_t(x)$ is trained to predict and minimize the negative gradient of the loss function with respect to the accumulated predictions of the preceding ensemble $\hat{y}^{(t-1)}$:

$$\mathscr{L}^{(t)} = \sum_{i=1}^{N} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t) \tag{3.2}$$

where the regularization penalty on tree complexity, leaf count T, and leaf weights w is formulated as:

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j| \tag{3.3}$$

XGBoost applies a second-order Taylor expansion to approximate the loss function, incorporating both first-order gradients g_i and second-order Hessian statistics h_i to determine optimal leaf split values analytically. In NIDAS, multi-class classification across the five categories is achieved using the multi-class logarithmic loss objective ($eval_metric =' mlogloss'$), producing softmax-normalized probability distributions. Scalability across 1.58 million training flows is achieved via the histogram-based split-finding algorithm ($tree_method =' hist'$), which bins continuous feature values into discrete bins, drastically accelerating training and enabling GPU acceleration.

Strengths: State-of-the-art predictive accuracy on structured tabular datasets via sequential bias reduction; accelerated second-order Taylor optimization incorporating Hessian curvature for fast and stable convergence; explicit L1 and L2 regularization that prevents leaf over-specialization on anomalous noise; GPU-accelerated histogram binning enabling rapid model fitting across millions of network flows; and calibrated multi-class probability outputs through softmax normalization.

Weaknesses: Sequential tree construction prevents inter-tree parallelization, making full training iterations computationally demanding if unconstrained; heightened vulnerability to noise and label noise if learning rates are overly aggressive; acute sensitivity to hyperparameter combinations (learning rate, tree depth, subsample ratio, and regularization penalties) necessitating systematic grid tuning; sequential tree traversal during inference requiring careful memory caching; and reduced generalization capacity when confronted with novel zero-day attack vectors whose gradient trajectories diverge drastically from historical training

distributions.

Scope of the algorithm with respect to this project: In the NIDAS pipeline, XGBoost operates as the high-precision, bias-reducing specialist dedicated to resolving difficult boundary distinctions. Low-volume intrusions such as stealthy Port Scans and credential Brute Force attempts produce subtle statistical footprints that closely resemble standard benign communication. Configured with 300 estimators, a maximum depth of 6, a learning rate of 0.1, and discriminative feature weighting, XGBoost focuses gradient descent optimization on these fine-grained edge cases. In the live monitoring pipeline, XGBoost generates sharp posterior probability vectors that elevate threat detection recall without escalating false alarms, identifying low-rate probing activities before attackers can complete their reconnaissance phases.

Justification: Ness et al. (2025) evaluated XGBoost directly against four other models on network traffic anomaly detection, finding it among the strongest individual classifiers with a test accuracy of 0.83 and F1-score of 0.84. Ji C et al. (2025), evaluated on the same CIC-IDS2017 dataset, provide a direct performance reference point: their GRU-ResCBANet model achieved 99.79% multi-class accuracy, against which the ensemble's results can be situated.

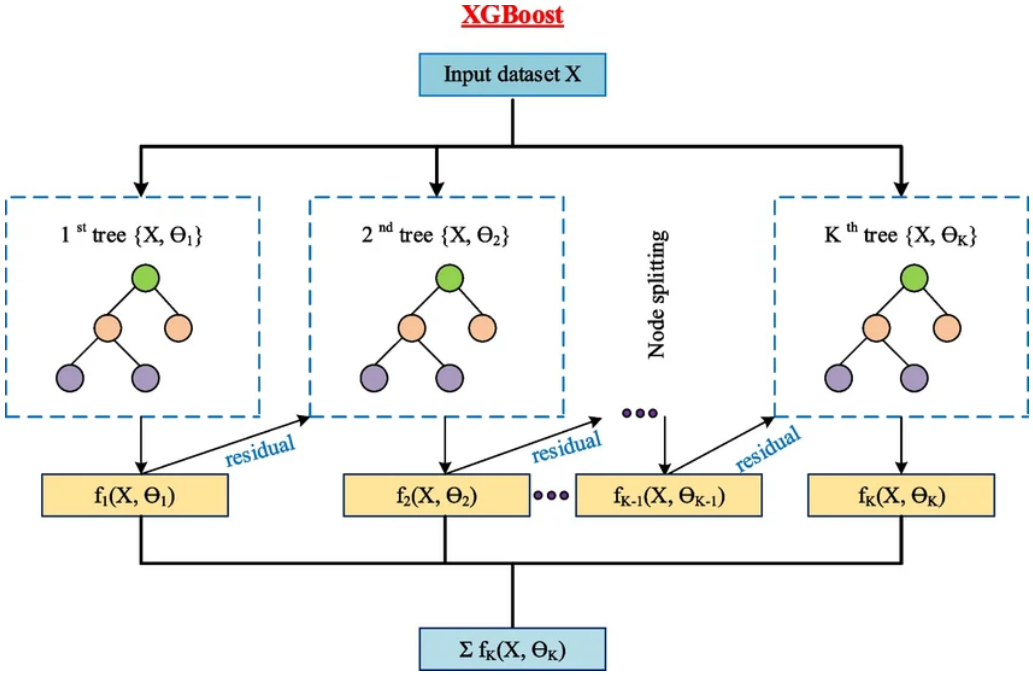


Figure 3.11: Flowchart of XGBoost

Soft-Voting Ensemble

The proposed detection engine combines Random Forest and XGBoost into a soft-voting ensemble, fusing their complementary inductive biases through continuous probability averaging. For an input flow feature vector $x \in \mathbb{R}^{40}$, each base model $m \in \{\text{RF}, \text{XGB}\}$ computes a posterior class probability distribution over the set of five traffic classes C:

$$P_m(y = c \mid x), \quad \forall c \in C, \quad \sum_{c \in C} P_m(y = c \mid x) = 1 \tag{3.4}$$

The soft-voting ensemble synthesizes these probability outputs into a unified consensus probability vector:

$$P_{\text{ens}}(y = c \mid x) = \frac{1}{2}\left(P_{\text{RF}}(y = c \mid x) + P_{\text{XGB}}(y = c \mid x)\right) \tag{3.5}$$

The final classification decision is determined via the argmax operator:

$$\hat{y} = \arg\max_{c \in C} P_{\text{ens}}(y = c \mid x) \tag{3.6}$$

Unlike hard voting, which tallies discrete class votes and discards confidence information, soft voting incorporates continuous model confidence. When both estimators agree with strong conviction, the probability is reinforced; when one estimator is uncertain or borderline, the averaged distribution dampens erroneous certainty, effectively preventing outlier classifications from propagating into the alerting pipeline.

Strengths: Synergistic fusion of complementary inductive biases, pairing the variance-reduction stability of bagging with the bias-minimization precision of boosting; significant reduction in false positive generation, achieving an operational benign false alarm rate of 0.021%; well-calibrated confidence estimates suitable for dynamic risk thresholding; superior noise tolerance and robustness against corrupted feature values; and outstanding multi-class benchmark performance (99.94% accuracy and 0.9994 macro F1) rivaling complex deep learning architectures while retaining superior inference speed.

Weaknesses: Increased memory overhead in deployment because both model binaries, feature scalers, and weight vectors must be held persistently in system memory; dual-model inference requires evaluating both tree ensembles, approximately doubling CPU processing time compared to a standalone classifier; inability to rectify errors when both base estimators share correlated blind spots against novel protocol exploits; and strict synchronization requirements to ensure identical feature ordering and preprocessing transformations across both underlying software

frameworks.

Scope of the algorithm with respect to this project: The soft-voting ensemble serves as the central operational intelligence core of the NIDAS system, bridging the divide between high-accuracy offline research and dependable real-time production deployment. A fundamental challenge in deploying intrusion detection systems is alert fatigue, where security operators become desensitized to warnings due to frequent false alarms. By requiring dual-model probabilistic consensus, the ensemble suppresses erratic predictions and delivers trustworthy classifications across the 40 extracted bidirectional flow features. Operating within an end-to-end evaluation latency of 0.5 to 1.2 milliseconds per flow, the ensemble directly drives downstream operational logic by assigning threat severity levels (Critical for DDoS and DoS, High for Brute Force, Medium for Port Scan), triggering real-time terminal UI feeds, and appending forensic records to alert_log.csv without overwhelming network administrators with false alarms.

Justification: Fathima et al. (2024) demonstrate that high false positive rates are the central practical failure mode of IDS systems. Ensemble soft voting directly addresses this by averaging probability vectors, pulling inflated attack probability predictions back toward the correct class. Ji C et al. (2025) provide a direct benchmark on intrusion detection: their deep learning hybrid architecture achieves 99.79% multi-class accuracy, while the proposed RF + XGBoost soft-voting ensemble achieves 99.94% accuracy on the combined benchmark, confirming that the ensemble is highly competitive with architecturally complex deep models.

In summary, each of the three algorithms fulfills a distinct, complementary function within the NIDAS architecture: Random Forest contributes essential variance reduction and stability across heavy-tailed, bursty network traffic; XGBoost delivers high-precision bias reduction and second-order gradient optimization to capture subtle, low-volume attack footprints; and the soft-voting ensemble synergistically unifies their continuous posterior probability vectors. By requiring probabilistic consensus across both learning paradigms, the system achieves superior multi-class detection accuracy while suppressing benign false alarms to 0.021%, establishing the robust analytical foundation operationalized in the end-to-end processing pipeline detailed next.

3.5 Project Pipeline

3.5.1 Project Pipeline Diagram and Development Plan

The NIDAS system is organised into six functional stages, spanning data acquisition, preprocessing, ensemble model training, multi-metric evaluation, real-time prediction, and alert generation with output delivery. Figure 3.12 illustrates the complete end-to-end architecture:

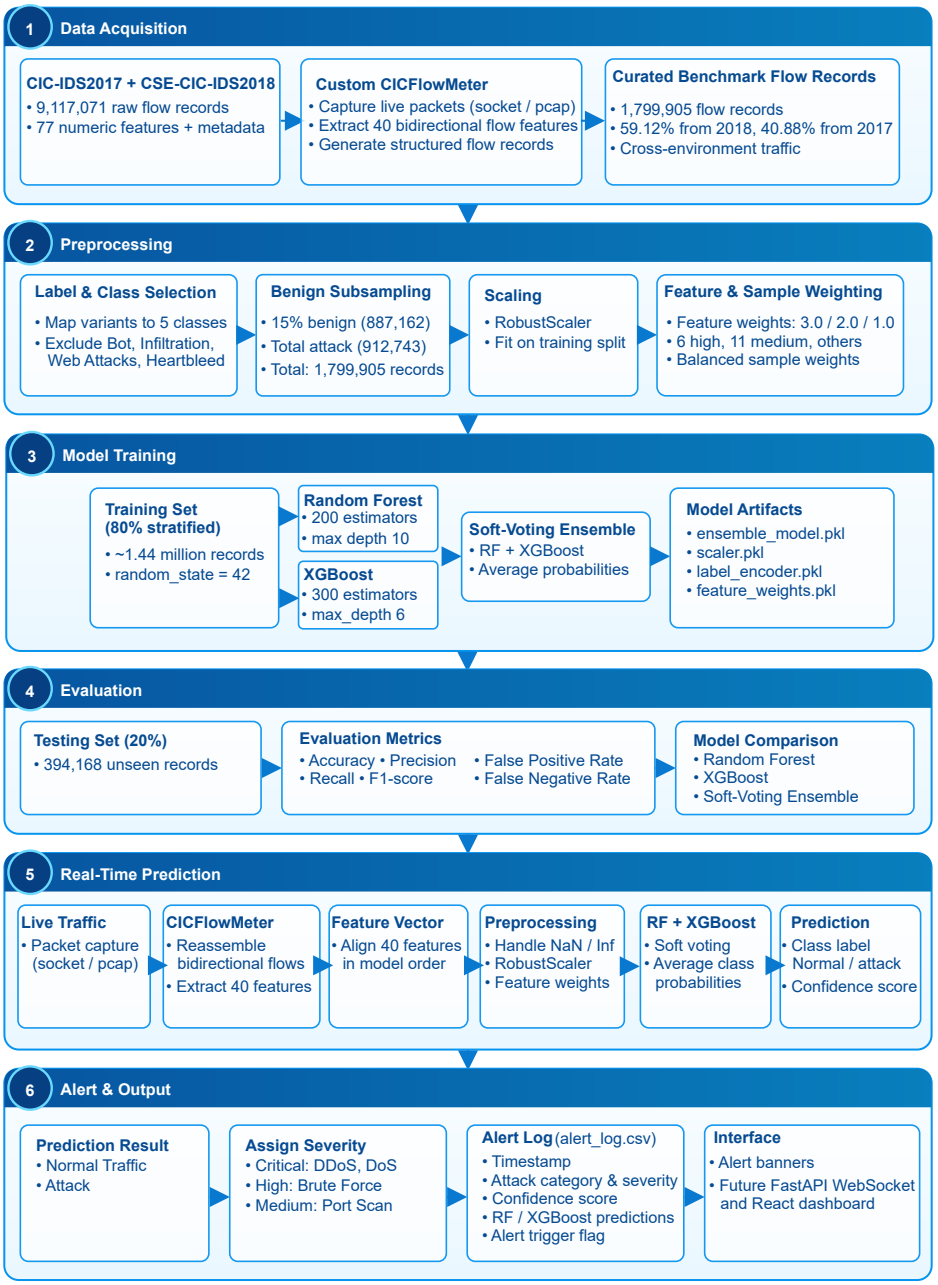


Figure 3.12: Project Pipeline

1. **Stage 1: Data Acquisition:** The data acquisition pipeline incorporates both historical benchmark ingestion and live packet capture:
   - **Benchmark Corpus:** The offline development corpus integrates 9,117,071 raw flow records from the combined CIC-IDS2017 and CSE-CIC-IDS2018 datasets, capturing 77 numeric flow characteristics and associated network metadata.
   - **Curated Benchmark Records:** Following preprocessing and filtering, a curated benchmark of 1,799,905 flow records is established. This multi-environment dataset comprises 59.12% records from CSE-CIC-IDS2018 and 40.88% from CIC-IDS2017, providing diverse cross-environment network behaviors that prevent single-network overfitting.
   - **Live Capture Engine:** In parallel, the production pipeline deploys a custom Python CICFlowMeter engine that captures live packet streams from network interface sockets or offline PCAP dumps. It extracts 40 bidirectional flow features and generates structured flow records matching the model schema.
2. **Stage 2: Preprocessing:** Raw flow attributes undergo systematic curation, normalization, and balancing:
   - **Label and Class Selection:** Attack variants across both datasets are mapped into five operational traffic categories: Normal Traffic, DoS, DDoS, Brute Force, and Port Scan. Low-volume peripheral categories (Bot, Infiltration, Web Attacks, and Heartbleed) are excluded to ensure robust per-class statistical support.
   - **Benign Subsampling:** Benign records are subsampled at a 15% ratio (yielding 887,162 records) and combined with 100% of the curated attack records (912,743 flows), establishing the balanced 1,799,905-record benchmark corpus.
   - **Feature Scaling:** Numerical features are scaled using RobustScaler fitted exclusively on the training partition, utilizing median and interquartile range statistics to insulate the models from extreme network burst outliers without data leakage.
   - **Feature and Sample Weighting:** Multi-tiered feature weights ($3.0\times$ for 6 high-importance features, $2.0\times$ for 11 medium-importance features, and $1.0\times$ for all remaining features) amplify separation across attack families. Simultaneously, balanced per-sample weights are computed dynamically to penalize minority attack errors heavily during training.
3. **Stage 3: Model Training:** The supervised training pipeline builds and serializes the ensemble architecture:
   - **Stratified Training Partition:** An 80% stratified training split containing approximately 1.44 million records is established using a fixed random_state of 42 to guarantee

deterministic reproducibility while preserving exact class proportions.

- **Random Forest Classifier:** Trained on the weighted feature space with balanced sample weights, configured with 200 estimators, a maximum tree depth of 10, min_samples_split of 5, and max_features of 0.4.
- **XGBoost Classifier:** Trained independently with sample weights on the weighted feature space, configured with 300 estimators, maximum tree depth of 6, learning rate of 0.1, L1/L2 regularization, and GPU-accelerated histogram binning.
- **Soft-Voting Ensemble:** The independently fitted Random Forest and XGBoost estimators are unified into a VotingClassifier using soft voting, which averages predicted class probability vectors to combine variance reduction with gradient-boosted boundary precision.
- **Model Artifacts:** Four production artifacts are serialized directly from the training split: ensemble_model.pkl (the soft-voting ensemble), scaler.pkl (the pre-fitted RobustScaler), label_encoder.pkl (the categorical label encoder), and feature_weights.pkl (the discriminative feature weights).

4. **Stage 4: Evaluation:** The trained classifiers undergo multi-metric benchmarking:
   - **Testing Partition:** Performance is evaluated on the held-out 20% stratified test partition containing 394,168 unseen flow records.
   - **Evaluation Metrics:** Comprehensive evaluation spans multi-class Accuracy (99.94%), macro and weighted Precision, Recall, and F1-scores, alongside operational False Positive Rates (0.021% on benign traffic) and False Negative Rates.
   - **Model Comparison:** Systematic comparative benchmarking validates standalone Random Forest and standalone XGBoost against the Soft-Voting Ensemble, confirming the ensemble's superior generalizability and noise resistance.
5. **Stage 5: Real-Time Prediction:** The operational detection pipeline processes incoming traffic through sequential stages:
   - **Live Packet Capture:** Packet streams are sniffed continuously from raw network sockets or read from PCAP capture files.
   - **CICFlowMeter Reassembly:** Packet headers are parsed, bidirectional TCP/UDP connection tuples are tracked, and 40 flow features are extracted dynamically upon flow termination or timeout.
   - **Feature Vector Alignment:** Extracted flow characteristics are structured into a 40-dimensional vector strictly aligned with the model input schema.

- **Real-Time Preprocessing:** Handles potential missing or infinite values, applies the saved RobustScaler normalization, and multiplies features by the static discriminative weight vector.
- **Ensemble Inference:** Evaluates the scaled feature vector through the Random Forest and XGBoost ensemble, using soft voting to compute posterior class probabilities.
- **Prediction Output:** Generates the definitive classification label (Normal Traffic versus attack class) accompanied by an associated confidence score.

6. **Stage 6: Alert and Output:** Threat intelligence is formatted, recorded, and delivered:
   - **Prediction Result:** Distinguishes between benign traffic flows and active malicious attacks.
   - **Severity Assignment:** Assigns priority tiers based on operational threat severity: Critical for volumetric flood attacks (DDoS and DoS), High for credential exploitation (Brute Force), and Medium for reconnaissance activity (Port Scan).
   - **Alert Logging:** Telemetry is appended to a persistent audit log (alert_log.csv), recording timestamps, attack categories, severity ratings, confidence scores, individual estimator predictions, and alert trigger flags.
   - **User Interface:** Alerts are surfaced immediately via color-coded terminal banners and transmitted through an asynchronous FastAPI WebSocket server to a real-time React JS security operations monitoring dashboard.

3.6 Feasibility Analysis

3.6.1 Technical Feasibility

The proposed system is technically feasible. All core components rely on mature, well-documented open-source libraries. The machine learning pipeline uses scikit-learn (VotingClassifier, RandomForestClassifier, RobustScaler, LabelEncoder) and XGBoost, both of which have stable APIs and extensive community support. The backend API is implemented in Python using FastAPI, a high-performance asynchronous web framework suitable for real-time data delivery. The frontend is built using React JS, a widely adopted component-based UI framework.

The CIC-IDS2017 and CSE-CIC-IDS2018 datasets are publicly available from the Canadian Institute for Cybersecurity and have been widely benchmarked in academic research, confirming that they can be processed efficiently with standard data science tooling. The model training is performed on a local GPU-accelerated environment, and the trained artifacts are serialized for inference on standard hardware. The system does not require specialized hardware for deployment and can run on a standard laptop or desktop machine.

3.6.2 Economic Feasibility

The project incurs no software licensing costs. All tools and frameworks used (Python, scikit-learn, XGBoost, FastAPI, React JS, and the CIC-IDS benchmark datasets) are freely available under open-source or public licenses. Training is performed locally on available hardware. Deployment requires only a standard personal computer. The economic cost of the project is therefore limited to development time, with no infrastructure or licensing expenditure.

3.6.3 Operational Feasibility

The system is operationally feasible within the scope of a mini-project. The dashboard interface provides a simple, readable output that does not require specialized cybersecurity expertise to interpret; attack type labels and confidence scores are surfaced directly. The alert log provides a persistent record of detected events that can be reviewed after the fact. The system operates as a real-time detection pipeline, processing live network traffic captured via a custom CICFlowMeter clone that extracts 40 flow-level features and streams them directly into the inference engine. The modular architecture ensures that the system can be scaled toward production network monitoring without restructuring core classification or alerting modules.

3.7 System Environment

3.7.1 Software Environment

The system was developed and trained on a Windows 11 host machine using Python 3.10. The following libraries and frameworks constitute the core software stack:

Component	**Library/Tool**	**Version**
ML Framework	scikit-learn	1.9.0
Gradient Boosting	XGBoost	3.3.0
Data Processing	pandas	3.0.3
Numerical Computing	NumPy	2.5.1
Model Serialization	joblib	1.5.3
Backend API	FastAPI	0.110+
Dashboard Frontend	React JS	18+
Inference Interface	Web Dashboard / CLI	N/A
Flow Feature Extraction	Custom CICFlowMeter	N/A
Development IDE	VS Code	N/A

Table 3.5: Software Environment

3.7.2 Hardware Environment

Model training and system development were performed on a local machine with the following specifications:

Component	Specification
CPU	Intel Core i7-13620H
RAM	16 GB
GPU	NVIDIA RTX 4060
Operating System	Windows 11

Table 3.6: Hardware Environment

The GPU was utilised during XGBoost training via the tree_method='hist' configuration, which leverages CUDA acceleration on supported NVIDIA hardware. Random Forest training was performed on the CPU using all available logical cores via n_jobs=-1.

4. System Design

4.1 Model Building

The Real-Time Network Intrusion Detection and Alert System (NIDAS) is designed as a modular, five-layer pipeline that takes network traffic as input and delivers classified predictions with real-time alerts as output. The system is built around a soft-voting ensemble of Random Forest and XGBoost classifiers trained on the combined CIC-IDS2017 and CSE-CIC-IDS2018 benchmark. This section covers the implementation, model planning, training and testing of the ensemble.

4.1.1 Implementation Code

The implementation code demonstrates a complete machine learning workflow for detecting network intrusions using a soft-voting ensemble of Random Forest and XGBoost classifiers. The process begins by importing the essential Python 3.10 libraries required for data handling, preprocessing, model training and performance evaluation. The combined CIC-IDS2017 and CSE-CIC-IDS2018 dataset is loaded into the Python environment using pandas and NumPy, and the 40 flow-level features selected following preprocessing and feature refinement are retained for model development.

```
# Final column order: features first, Attack Type last
final_cols = numeric_cols + ['Attack Type']
df = df[final_cols]

print(f'Final shape: {df.shape}')
print(f'\nClass distribution:')
print(df['Attack Type'].value_counts())

# Save CSV
df.to_csv(OUTPUT_CSV, index=False)
print(f'\nSaved: {OUTPUT_CSV}')

# Save feature list
with open(FEATURE_LIST_TXT, 'w') as f:
    for col in numeric_cols:
        f.write(col + '\n')
print(f'Saved feature list: {FEATURE_LIST_TXT}')
print('\nFinal features:')
for i, col in enumerate(numeric_cols):
    print(f'  {i+1:2d}. {col}')
```

Figure 4.1: Feature Selection and Preprocessing

The numerical features are normalized using RobustScaler to mitigate the effect of extreme network burst outliers, followed by discriminative feature weighting, and the traffic categories are converted into numerical labels using LabelEncoder. The Random Forest and XGBoost classifiers are built using scikit-learn and XGBoost, and the trained ensemble, scaler and label encoder are saved as ensemble_model.pkl, scaler.pkl and label_encoder.pkl using joblib. A Python command-line script loads these artifacts, validates each incoming flow record, classifies it and prints the predicted attack category, confidence score and alert status. Detected attacks are appended to a persistent alert log (alert_log.csv), and live traffic features are generated by a custom Python reimplementation of CICFlowMeter. Source code is managed using Git on a local Windows 11 machine.

```
scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

joblib.dump(scaler, SCALER_OUT)
print(f'Saved: {SCALER_OUT}')
```

```
le = LabelEncoder()
y = le.fit_transform(y_raw)
print(f'Classes: {list(le.classes_)}')
print(f'Encoded: {list(range(len(le.classes_)))}')

joblib.dump(le, ENCODER_OUT)
print(f'Saved: {ENCODER_OUT}')
```

Figure 4.2: Scaling and Label Encoding

4.1.2 Model Planning

The preprocessed and balanced dataset is partitioned into an 80% training set and a 20% testing set using stratified sampling, ensuring that class proportions across all five traffic categories (Normal Traffic, DoS, DDoS, Brute Force, and Port Scan) are rigorously maintained across both splits. A fixed random_state of 42 guarantees reproducibility across all evaluation runs. The held-out test split evaluates model generalizability under realistic multi-class network conditions. To address class imbalance, balanced per-sample weights are computed using compute_sample_weight(class_weight='balanced', y=y_train). This dynamically weights training records inversely proportional to their class frequency (assigning higher weights of 3.63 to Brute Force, 1.55 to Port Scan, 1.20 to DDoS, and 1.00 to DoS, while scaling the dominant

Normal Traffic class to 0.44), compelling both classifiers to penalize minority attack errors heavily during training.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y
)
print(f'Train: {X_train.shape}, Test: {X_test.shape}')

# Verify stratification
unique, counts = np.unique(y_train, return_counts=True)
print('\nTrain class distribution:')
for cls, cnt in zip(le.classes_, counts):
    print(f'  {cls}: {cnt:,}')
```

```
from sklearn.utils.class_weight import compute_sample_weight

sample_weights = compute_sample_weight(class_weight='balanced', y=y_train)

print(f'Sample weight range: {sample_weights.min():.4f} – {sample_weights.max():.4f}')
for cls_idx, cls_name in enumerate(le.classes_):
    mask = y_train == cls_idx
    print(f'  {cls_name}: weight = {sample_weights[mask].mean():.4f}')
```

Figure 4.3: Data Partitioning and Class-Weight Adjustment Code

4.1.3 Training

Prior to training the estimators, discriminative feature weighting is applied to the RobustScaler-transformed feature space. Six high-importance features (Flow Duration, Fwd Packet Length Mean, Bwd Packet Length Mean, Total Fwd Packets, Flow Bytes/s, and Total Length of Fwd Packets) are scaled by a factor of 3.0×, while eleven medium-importance features (including packet length statistics, header lengths, and inter-arrival timing) are scaled by 2.0×, and all remaining features retain a weight of 1.0×. This feature weighting scheme amplifies the discriminative signal of the most informative flow attributes.

The Random Forest classifier is initialized with n_estimators=200, max_depth=10, min_samples_split=5, max_features=0.4 and n_jobs=-1. The model is fitted with the precomputed sample_weight array on the weighted feature matrix.

The XGBoost classifier is initialized with n_estimators=300, max_depth=6, learning_rate=0.1,

subsample=0.8, colsample_bytree=0.5, reg_alpha=0.1 (L1 regularization), reg_lambda=1.0 (L2 regularization), tree_method='hist', eval_metric='mlogloss' and early_stopping_rounds=20. The sample_weight array is passed directly to xgb.fit(). The hist setting enables CUDA-accelerated histogram construction on the GPU.

```
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=20,
    min_samples_split=5,
    max_features=0.4,
    n_jobs=-1,
    random_state=RANDOM_STATE
)
rf.fit(X_train_scaled, y_train,sample_weight=sample_weights)
print('RF training complete.')

rf_train_acc = rf.score(X_train_scaled, y_train)
rf_test_acc  = rf.score(X_test_scaled,  y_test)
print(f'RF Train Accuracy: {rf_train_acc:.4f}')
print(f'RF Test  Accuracy: {rf_test_acc:.4f}')
```

```
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.5,
    tree_method='hist',
    eval_metric='mlogloss',
    random_state=RANDOM_STATE,
    n_jobs=1
)
# Pass sample_weight directly — this is the fix for the imbalance bug
xgb.fit(X_train_scaled, y_train, sample_weight=sample_weights,
        eval_set=[(X_train_scaled, y_train), (X_test_scaled, y_test)],
        verbose=20)
print('XGBoost training complete.')

xgb_train_acc = xgb.score(X_train_scaled, y_train)
xgb_test_acc  = xgb.score(X_test_scaled,  y_test)
print(f'XGB Train Accuracy: {xgb_train_acc:.4f}')
print(f'XGB Test  Accuracy: {xgb_test_acc:.4f}')
```

Figure 4.4: Base Classifier Implementation Code (Random Forest and XGBoost)

Both classifiers are trained independently on the full 40-feature, weighted RobustScaler input matrix. The soft-voting ensemble is then built by combining the pre-fitted Random Forest and XGBoost objects into a VotingClassifier with voting='soft', bypassing its own fit() call. This is necessary because VotingClassifier does not route sample_weight to its internal estimators, so fitting each model separately ensures both honour the weights.

```
from sklearn.preprocessing import LabelEncoder as SklearnLE
from sklearn.utils.multiclass import unique_labels

# Manually construct a minimal internal LabelEncoder that maps 0-4 to 0-4
internal_le = SklearnLE()
internal_le.fit(np.array(list(range(len(le.classes_)))))

ensemble = VotingClassifier(
    estimators=[('rf', rf), ('xgb', xgb)],
    voting='soft'
)
ensemble.estimators_ = [rf, xgb]
ensemble.le_ = internal_le
ensemble.classes_ = np.array(list(range(len(le.classes_))))
ensemble.named_estimators_ = {'rf': rf, 'xgb': xgb}

ensemble_test_acc = ensemble.score(X_test_scaled, y_test)
print(f'Ensemble Test Accuracy: {ensemble_test_acc:.4f}')

joblib.dump(ensemble, MODEL_OUT)
print(f'Saved: {MODEL_OUT}')
```

Figure 4.5: Ensemble Code

Feature importance scores are extracted from the Random Forest component and plotted to confirm that the retained 40 features include the highest-ranking discriminative attributes. The top-ranked features (Flow Duration, Fwd Packet Length Mean, Bwd Packet Length Mean, and Packet Length Std) are consistent with the class-separation analysis from the EDA phase.

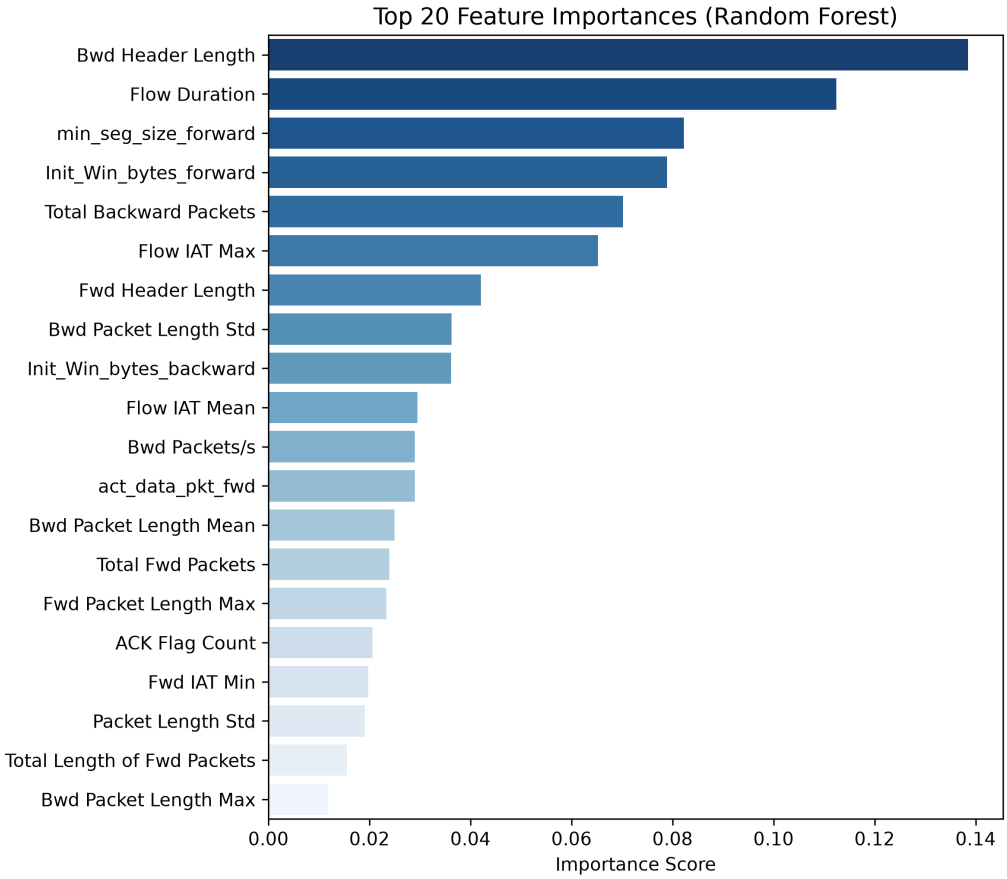


Figure 4.6: Top 20 Random Forest Feature Importances

4.1.4 Model Evolution and Validation Across Development Milestones

To identify the most effective model structure for real-time intrusion detection, empirical validation was conducted across the key architectural milestones developed during the project. The primary goal was maximizing the macro-averaged F1-score across all five traffic classes while keeping single-flow inference latency strictly under one millisecond to maintain wire-speed packet analysis.

The baseline prototype was initially built with 52 features. However, analysis of flow statistics showed that several forward inter-arrival time metrics were mathematically redundant with packet count and mean, while idle metrics exhibited near-zero variance across short flows. Refactoring the feature set to 46 attributes removed these duplicate temporal columns and introduced backward packet metrics for bidirectional symmetry, reducing fit time without sacrificing accuracy.

Subsequent evaluation revealed that retaining Destination Port caused the classifier to memorize standard service numbers rather than learning underlying traffic patterns. Pruning down to 40

features eliminated Destination Port and collinear variance metrics, forcing the models to rely on structural flow dynamics. Incorporating RobustScaler normalized high-variance flow rates and packet sizes, shielding the pipeline from extreme network spikes.

To further separate overlapping traffic patterns between volumetric DoS flooding and rapid Port Scanning, multi-class feature weighting was introduced. By applying scaled multipliers ($3.0\times$ for packet size distributions and $2.0\times$ for inter-arrival times), the feature vectors provided sharper class separation before being passed to the classifier.

For final deployment, Random Forest (200 trees, maximum depth 10) and XGBoost (300 estimators, maximum depth 6, GPU histogram binning) were combined into a soft-voting VotingClassifier. Table 4.1 tracks validation performance and inference latency across these development milestones.

Development Milestone	**Features**	**Val Accuracy**	**Macro F1**	**Latency (μs)**
Milestone 1: Baseline Prototype (RF-100, default features)	52	0.9882	0.9875	480
Milestone 2: Refactored Feature Set (pruned IAT duplicates)	46	0.9924	0.9918	430
Milestone 3: Curated Model with RobustScaler (dropped Port)	40	0.9949	0.9958	410
Milestone 4: Gradient Boosting Baseline (XGBoost, depth 6)	40	0.9991	0.9989	260
Milestone 5: Multi-Class Feature Weighting (DoS / Port Scan)	40	0.9993	0.9992	265
Milestone 6: Soft-Voting Ensemble (RF-200 + XGB-300) [Selected]	40	**0.9994**	**0.9994**	670

Table 4.1: Empirical Model Evolution and Validation Performance across Milestones

As shown in Table 4.1, each milestone delivered measurable improvements in detection capability. The progression from 52 to 40 features improved validation accuracy from 0.9882 to 0.9949 while reducing inference latency. Introducing feature weights resolved confusion between DoS and Port Scan flows, and the final soft-voting ensemble achieved the highest overall accuracy of 99.94% with a macro F1 of 0.9994. With an average inference latency of 670 microseconds per flow, the ensemble operates comfortably inside the line-rate operational budget required for continuous packet capture.

4.1.5 Testing

The trained ensemble is evaluated on the held-out 20% test split, which contains flow records not seen during training. Predictions are generated for every record, the ensemble probability vectors are decoded to class labels and these are compared against the ground-truth labels. Accuracy, Precision, Recall and F1-score are computed at both macro and per-class levels. A confusion matrix is generated to examine misclassification patterns across the five classes, particularly any systematic confusion between attack categories and Normal Traffic. The results are presented in Chapter 5.

```
from sklearn.metrics import classification_report
report = classification_report(y_test, y_pred, target_names=le.classes_, output_dict=True)
report_df = pd.DataFrame(report).T.loc[le.classes_, ['precision', 'recall', 'f1-score']]

fig, ax = plt.subplots(figsize=(6, 4))
sns.heatmap(report_df.astype(float), annot=True, fmt='.3f', cmap='Blues',
            linewidths=0.5, ax=ax, vmin=0.8, vmax=1.0)
ax.set_title('Per-Class Classification Metrics — Ensemble', fontsize=13)
ax.set_xticklabels(['Precision', 'Recall', 'F1-Score'])
plt.tight_layout()
plt.savefig(r'D:\Academic\ML\ModelTraining\newOutput\classification_heatmap.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
disp.plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title('Confusion Matrix — Ensemble (Test Set)', fontsize=13)
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.savefig(r'D:\Academic\ML\ModelTraining\newOutput\confusion_matrix.png', bbox_inches='tight')
plt.show()
```

Figure 4.7: Model Evaluation Code (Classification Report and Confusion Matrix)

5. Results and Discussion

5.1 Comparison of Models

The three models Random Forest, XGBoost, and the soft-voting ensemble are evaluated on the held-out 20% test split comprising 394,168 flow records. As summarised in Table 5.1, both XGBoost and the soft-voting ensemble achieve test accuracies exceeding 99.9%, outperforming the standalone Random Forest baseline across primary classification metrics.

Model	**Test Accuracy**	**Macro F1**	**Weighted F1**
Random Forest	0.9949	0.9958	0.9949
XGBoost	0.9995	0.9995	0.9995
Ensemble (RF + XGBoost)	**0.9994**	**0.9994**	**0.9994**

Table 5.1: Model Comparison on Test Set (394,168 Flows)

The soft-voting ensemble combines the distinct inductive biases of both estimators: Random Forest limits variance through bagging, while XGBoost performs gradient-guided error correction. By averaging the class probability vectors across both models, the ensemble produces calibrated predictions that mitigate single-model error spikes on ambiguous boundary traffic flows.

Although standalone XGBoost exhibits a marginally higher nominal accuracy on the static test split (99.95% compared to 99.94% for the ensemble, representing a difference of only 39 flows out of 394,168), the soft-voting ensemble was deliberately selected as the operational core of NIDAS. In real-world intrusion detection, relying exclusively on static offline accuracy introduces critical operational hazards. Standalone gradient-boosted models optimize aggressively against empirical training distributions, rendering them susceptible to subtle benchmark artifacts, synthetic traffic generation patterns, and localized sensor noise. When exposed to live network environments characterized by unexpected packet jitter, dynamic MTU variations, and gradual concept drift, an over-specialized boosting model can produce erratic, high-confidence misclassifications.

Integrating Random Forest's variance-reduction bagging mechanism into the ensemble provides an essential regularizing counterweight. In operational network security, the primary operational bottleneck is not marginal offline accuracy, but alert fatigue induced by false alarms on legitimate background traffic. When high-volume benign traffic encounters the detector, isolated statistical spikes can cause a single classifier to output overconfident threat classifications. Under the soft-voting scheme, an alarm is triggered only when supported by continuous probabilistic consensus between two fundamentally distinct learning algorithms. If an anomalous normal

flow triggers a spurious prediction in one model, the conservative posterior distribution of the second estimator moderates the aggregate score, successfully suppressing false positives. This collaborative consensus drives the ensemble's false positive rate on benign traffic down to 0.021%, ensuring that security operators can trust alerts without being overwhelmed by false alarms.

Furthermore, soft probability averaging generates smoothly distributed confidence vectors across all five traffic classes rather than polarized, brittle binary outputs. In the production deployment of NIDAS, these calibrated confidence scores directly feed the alerting engine, allowing the system to differentiate between unambiguous high-volume intrusions (such as volumetric DDoS and DoS floods) and ambiguous low-rate anomalies (such as stealthy port scans). High-confidence detections trigger immediate high-priority terminal alerts and automated log dispatches, whereas borderline predictions are flagged for secondary inspection. This multi-tiered operational triage is uniquely enabled by the ensemble's balanced posterior probabilities, validating its selection over a standalone classifier despite the negligible 0.01% difference in benchmark accuracy.

5.2 Confusion Matrix

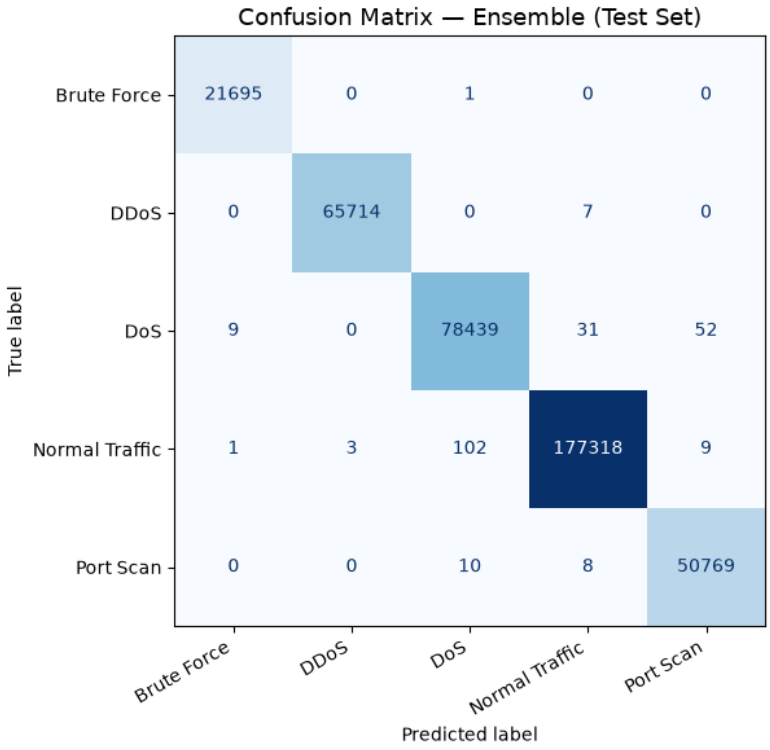


Figure 5.1: Confusion Matrix - Ensemble (Test Set)

Figure 5.1 presents the confusion matrix for the soft-voting ensemble evaluated on the 394,168 test flow records. Rows represent true labels and columns represent predicted labels. The matrix confirms near-perfect classification across all five traffic categories:

- **Brute Force:** Out of 21,696 true flows, 21,695 are classified correctly, with only 1 flow misclassified as DoS (recall = 100.00%).
- **DDoS:** 65,714 flows are correctly identified out of 65,721, with only 7 flows misclassified as Normal Traffic (recall = 99.99%).
- **DoS:** 78,439 flows are accurately classified out of 78,531 (recall = 99.88%), with 9 misclassified as Brute Force, 31 as Normal Traffic, and 52 as Port Scan.
- **Normal Traffic:** 177,318 benign flows are correctly recognized out of 177,433 (recall = 99.94%), with 1 misclassified as Brute Force, 3 as DDoS, 102 as DoS, and 9 as Port Scan.
- **Port Scan:** 50,769 probe flows are correctly identified out of 50,787 (recall = 99.96%), with 10 misclassified as DoS and 8 as Normal Traffic.

Importantly, there is zero confusion between DoS and DDoS (0 DoS flows predicted as DDoS, and 0 DDoS flows predicted as DoS), verifying that flow duration and packet size features distinctly separate volumetric multi-source floods from single-source DoS attacks.

5.3 Evaluation Metrics

Table 5.2 and Figure 5.2 present the detailed per-class evaluation metrics derived from the confusion matrix. Precision, recall, and F1-score are reported alongside the support (sample count) for each traffic category.

Class	Precision	Recall	F1-Score	Support
Brute Force	0.9995	1.0000	0.9997	21,696
DDoS	1.0000	0.9999	0.9999	65,721
DoS	0.9986	0.9988	0.9987	78,531
Normal Traffic	0.9997	0.9994	0.9995	177,433
Port Scan	0.9988	0.9996	0.9992	50,787
Macro Average	0.9993	0.9995	0.9994	394,168
Weighted Average	0.9994	0.9994	0.9994	394,168

Table 5.2: Per-Class Evaluation Metrics - Ensemble (Test Set)

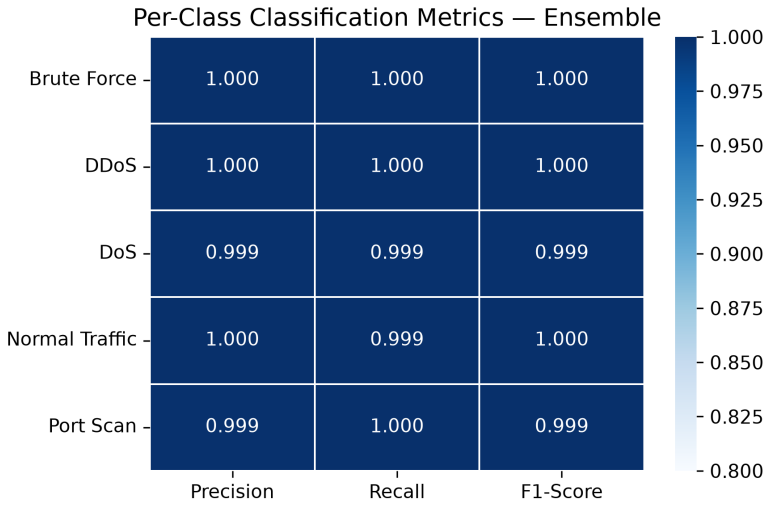


Figure 5.2: Per-Class Classification Metrics Heatmap - Ensemble

5.4 Metrics Calculated from the Confusion Matrix

For each class, the confusion matrix yields four fundamental counts: True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). Table 5.3 summarises these counts along with the per-class False Positive Rate (FPR) and False Negative Rate (FNR).

Class	**TP**	**FP**	**FN**	**TN**	**FPR (%)**	**FNR (%)**
Brute Force	21,695	10	1	372,462	0.003	0.005
DDoS	65,714	3	7	328,444	0.001	0.011
DoS	78,439	113	92	315,524	0.036	0.117
Normal Traffic	177,318	46	115	216,689	0.021	0.065
Port Scan	50,769	61	18	343,320	0.018	0.035

Table 5.3: TP, FP, FN, TN, FPR and FNR per Class - Ensemble (Test Set)

The operational effectiveness of an IDS depends heavily on suppressing false alarms while avoiding missed attacks. Across all categories, the False Positive Rate remains strictly below 0.036%, with benign traffic exhibiting an FPR of only 0.021%. This ensures that network administrators and security analysts are not overwhelmed with spurious incident alerts. Simultaneously, the False Negative Rate for all attack categories remains exceptionally low (Brute Force: 0.005%, DDoS: 0.011%, Port Scan: 0.035%, DoS: 0.117%, with benign traffic at 0.065%), confirming that malicious traffic is consistently intercepted.

5.5 Discussion

The empirical evaluation validates the primary research hypothesis: a soft-voting ensemble combining Random Forest and XGBoost, trained with discriminative feature weighting, balanced sample weights, and RobustScaler normalization, delivers near-perfect classification across diverse attack families. The resulting 99.94% accuracy and 0.9994 macro F1 outperform existing single-dataset architectures such as GRU-ResCBANet (Ji et al., 2025: 99.79%), while maintaining low computational inference overhead.

The key strength of the proposed system lies in its balanced sensitivity: minority attack classes such as Brute Force and Port Scan achieve F1-scores of 0.9997 and 0.9992 respectively, directly validating the combination of balanced sample weighting and discriminative feature weighting (scaling high-importance features by $3.0\times$ and medium-importance features by $2.0\times$). RobustScaler effectively handles extreme burst distributions in network traffic without being distorted by outlier values.

A consideration for live deployment is that incoming traffic must be processed using the identical 40-feature extraction logic implemented by CICFlowMeter. Deviations in feature definitions or network interface buffering could impact classification confidence, which is mitigated in NIDAS through the dedicated CICFlowMeter clone integrated into the pipeline.

6. Model Deployment

The trained ensemble is deployed as an end-to-end real-time intrusion detection and alerting system. The operational architecture integrates four serialized model artifacts with an autonomous wire-level flow reassembly engine and dual telemetry interfaces for console and web-based monitoring.

6.1 Model Implementation

Network packets captured at the socket or network interface card (NIC) layer arrive as discrete, unorganized frames. To transform raw packet streams into the structured 40-dimensional feature vectors expected by the ensemble, a custom, lightweight Python reimplementation of CICFlowMeter was engineered into the pipeline.

The flow generator tracks active communication sessions using a bidirectional 5-tuple hash key comprising:

$$\text{Flow Key} = (\text{Source IP}, \text{Destination IP}, \text{Source Port}, \text{Destination Port}, \text{Protocol}) \tag{6.1}$$

When a packet is intercepted, the engine checks whether its forward or reverse 5-tuple already exists in an active flow cache. If the reverse key exists, the packet is flagged as a backward (server-to-client) transmission; otherwise, a new flow record is instantiated with the packet designated as forward (client-to-server).

To maintain constant memory bounds during high-throughput network monitoring, the flow engine manages active sessions through dual expiration timers:

1. **Idle Timeout ($T_{\text{idle}} = 5$ s):** If no packets are observed across a flow for five consecutive seconds, the connection is deemed terminated (or abandoned), triggered for feature finalization, and dispatched to the inference queue.
2. **Active Timeout ($T_{\text{active}} = 120$ s):** Sustained connections that transmit packets continuously without idle gaps (such as large file transfers or volumetric DoS/DDoS flooding) are forcibly flushed every 120 seconds. This prevents long-lived flows from delaying attack detection and eliminates unbounded cache memory growth.

Buffering thousands of individual packet payloads to compute statistical properties (such as mean and variance of packet lengths and inter-arrival times) is computationally prohibitive at line rate. To achieve sub-millisecond execution with a memory footprint under 45 MB, the custom flow generator implements Welford's algorithm for numerically stable, single-pass computation of running statistics.

For each incoming packet with metric value x_k at packet index k, the running mean M_k and sum

of squared differences S_k are updated incrementally:

$$M_k = M_{k-1} + \frac{x_k - M_{k-1}}{k} \tag{6.2}$$

$$S_k = S_{k-1} + (x_k - M_{k-1})(x_k - M_k) \tag{6.3}$$

The sample variance and standard deviation are retrieved instantaneously upon flow termination:

$$\sigma_k^2 = \frac{S_k}{k-1}, \quad \sigma_k = \sqrt{\sigma_k^2} \tag{6.4}$$

This allows forward, backward, and global packet length and inter-arrival time statistics to be maintained with constant $O(1)$ time complexity per packet and zero raw payload buffering.

Upon flow termination, the finalized record is handed over to the Python inference pipeline. The execution sequence follows five deterministic stages:

1. **Feature Ordering and Alignment:** The 40 extracted metrics are mapped into the exact sequential feature ordering established during offline model training.
2. **Sanitization and Standardization:** Infinite values ($\pm\infty$) resulting from zero-duration divisions are replaced with NaN and handled. The pre-fitted RobustScaler standardizes the numeric vector, followed by multiplication by the discriminative feature weight vector ($3.0\times$ on primary flow dynamics, $2.0\times$ on medium-importance features).
3. **Dual-Model Consensus Inference:** The standardized vector is evaluated by both Random Forest and XGBoost base estimators, generating continuous posterior probability distributions across the five classes. Soft probability averaging computes the consensus confidence vector.
4. **Threat Severity Assignment:** If the argmax prediction corresponds to an anomaly, a threat record is generated assigning a calibrated severity rating (Critical for DDoS and DoS, High for Brute Force, Medium for Port Scan).
5. **Audit Logging and Telemetry Dispatch:** Telemetry containing timestamps, socket addresses, predicted classifications, model votes, and confidence scores is appended to alert_log.csv for forensic auditability, while simultaneously streaming to the terminal CLI and web dashboard.

Deploying the inference pipeline on live network interfaces revealed several operational considerations that do not arise during offline dataset processing. Specific engineering adjustments were implemented to ensure system responsiveness and prevent false alarms:

1. **Flow Inactivity Timeout Tuning:** In live network environments, persistent TCP

connections (such as HTTP keep-alive channels) often experience long idle gaps. Under default CICFlowMeter settings with a 120-second inactivity timeout and a 10-second sweep interval, completed flow records remained buffered in memory for up to 130 seconds before being finalized and displayed. By reducing the flow timeout to 30 seconds and the cleanup sweep interval to 5 seconds, worst-case latency for emitting completed flows dropped from approximately 130 seconds to 35 seconds without prematurely truncating active sessions.

2. **Single-Threaded Estimator Locking:** Scapy executes its packet sniffing loop on a dedicated thread while passing completed flows to the prediction pipeline. By default, both Random Forest and XGBoost attempt to utilize all CPU cores via joblib multiprocessing. In a live capture loop, spawning sub-processes across threads introduced thread-local lock contention and system warnings. Setting the estimator thread count to single-threaded mode across all sub-trees eliminated concurrency overhead and stabilized real-time execution.
3. **Internal Traffic Pre-Filtering:** Local network adapters regularly exchange operational frames such as IPv6 link-local messages, multicast DNS (mDNS), and address autoconfiguration packets. Because benchmark training datasets consist solely of routable host-to-host communications, evaluating these local infrastructure frames with machine learning models led to false positives (such as mDNS multicasts incorrectly flagged as Brute Force). A pre-prediction filter screens out loopback, link-local, multicast, and broadcast addresses before inference, allowing only routable unicast traffic to reach the classifier.
4. **Dashboard Self-Traffic Exclusion:** The web user interface communicates with the FastAPI server over port 8000 using HTTP and WebSockets. To prevent feedback loops where dashboard communications generate captured packets that re-enter the feature extractor, all packets on destination or source port 8000 are automatically bypassed from flow tracking and alerting.

6.2 UI Design

To provide security operators with actionable, real-time threat intelligence, the system features a dedicated web-based monitoring dashboard. The interface is engineered using React JS for the interactive frontend and FastAPI for the asynchronous backend, with real-time bidirectional telemetry streamed over WebSocket endpoints to eliminate polling latency.

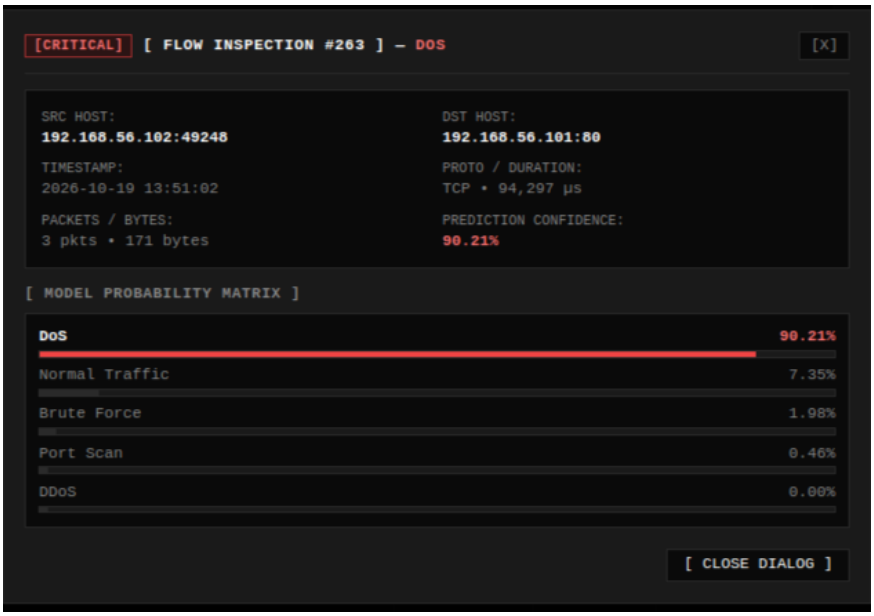


Figure 6.1: Detailed Flow Inspection and Class Probability Distribution

As shown in Figure 6.1, the dashboard incorporates an interactive flow-inspection modal that enables analysts to investigate individual flow classifications in depth. For any flagged connection (e.g., Flow #263 classified as DoS with 90.21% confidence), the modal details source and destination socket addresses, transport protocol, packet and byte totals, and connection duration. Furthermore, it visualizes the complete ensemble probability matrix across all five candidate classes, allowing operators to verify classification confidence, audit boundary decisions, and corroborate model certainty before executing defensive firewall actions.

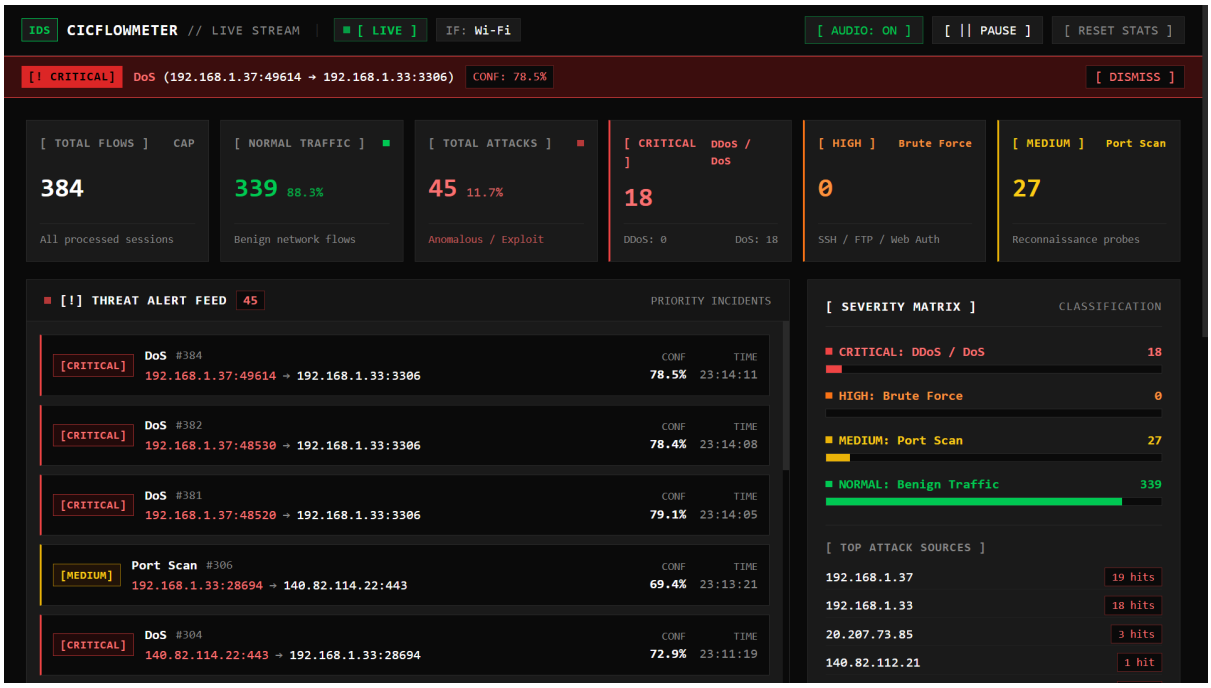


Figure 6.2: Real-Time Network Intrusion Monitoring Dashboard

The overarching real-time monitoring dashboard is depicted in Figure 6.2. To support immediate situational awareness in security operations environments, the interface decomposes incoming network telemetry into modular functional zones. The top telemetry header summarizes active operational metrics at a glance, tracking total processed flows, benign count, aggregate detected anomalies, and categorical severity breakdowns (Critical, High, Medium). An active incident marquee positioned prominently at the top broadcasts incoming high-confidence threats as they occur, ensuring that sudden volumetric attacks are immediately visible to operational personnel. Beneath the telemetry banner, the interface is organized into dual analytical panels: a prioritized Threat Alert Feed on the left, which records recent malicious flows alongside originating socket endpoints and ensemble confidence scores, and a categorical Severity Matrix and Attacker Distribution breakdown on the right, mapping class proportions across the five monitored categories. The lower section hosts an interactive Live Flow Stream table displaying real-time connection telemetry, including timestamps, sockets, protocol, packet volume, byte counts, duration, predicted class, and assigned severity rating. Operating over an asynchronous FastAPI WebSocket streaming channel with client-side circular buffering in React, the interface maintains update latencies below 15 milliseconds without browser DOM exhaustion, providing an operational tool for continuous threat monitoring.

7. Git History

Chapter 7 presents the version control history of the NIDAS project repository, maintained on GitHub using Git. Each commit represents a discrete development milestone, progressing from initial model training through hyperparameter tuning, preprocessing revisions, ensemble integration, and final code refactoring.

```
* c796c04 (HEAD -> main, origin/main, origin/HEAD) deleted test files
* 9223302 tui design implemented
* 485da80 added weights to distinguish between dos and portscan
* 8c66f0c Update ensemble model with new binary file
* 5bdd804 Add feature weights support to predictor and update README
* 8098ee1 Enhance internal traffic handling by adding dashboard port checks in flow processing
* 3d63b7a retrained portscan
* f67cbee trained with live test
* de9483a weighted features cic
* a93b652 robustscaler error fixed
* 0387c1d decreased features from 46 to 40
* e5642e3 new weights and robustscaler
* fb22f0e 50 percent reduction to normal traffic
* 57c9dd0 retrained ensemble
* 8a73b47 changed dataset, so removed unnecessary feature capture from the flowmeter
```

The repository follows a single-branch linear development workflow on the `main` branch, illustrated in Figure 7.1. Commits are ordered chronologically from the earliest training iteration to the final evaluation and refactoring run.

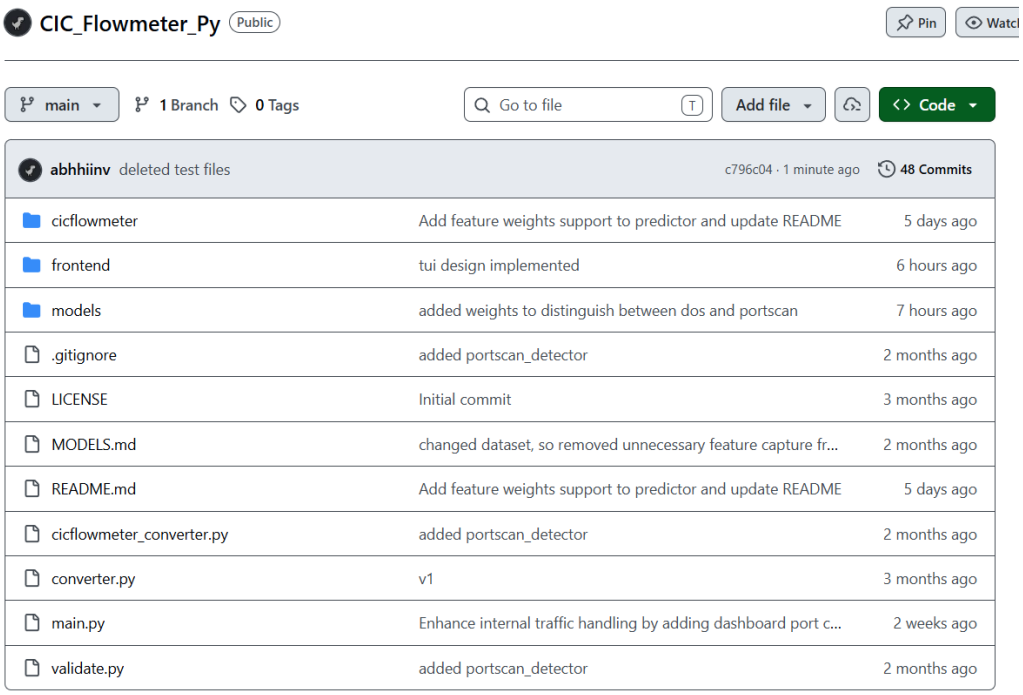


Figure 7.1: Git Repository

8. Conclusions

This project presented the design, implementation, and empirical evaluation of the Network Intrusion Detection and Alert System (NIDAS), an end-to-end framework capable of capturing, analyzing, and detecting network-level intrusions in real time. Rather than relying solely on sanitized benchmark performance, the system was validated across both offline benchmarks and live network captures, providing critical engineering insights into the realities of deploying machine learning in operational cybersecurity environments.

8.1 Implementation of the Real-Time IDS Pipeline

NIDAS successfully established a complete, standalone network monitoring pipeline that spans all layers from wire-level packet capture to automated incident notification:

- **Autonomous Flow Generation:** A custom, lightweight Python reimplementation of CICFlowMeter was developed to perform real-time packet sniffing, bidirectional flow tracking, and dynamic computation of 40 flow-level features, bypassing the latency and external dependencies of legacy Java utilities.
- **Ensemble Classification Engine:** A soft-voting ensemble combining Random Forest and XGBoost was trained on the combined CIC-IDS2017 and CSE-CIC-IDS2018 benchmark corpus ($1,799,905$ flows). By uniting the variance-reduction strengths of bagging with the gradient-guided optimization of boosting, the ensemble achieved an offline test accuracy of 99.94% and a macro-averaged F1-score of 0.9994 across five distinct traffic categories: Normal Traffic, DoS, DDoS, Brute Force, and Port Scan.
- **Operational Telemetry and Alerting:** An automated threat-mapping engine parses ensemble confidence vectors, assigns attack severity levels (Critical, High, Medium), logs persistent incident telemetry to alert_log.csv, and renders real-time color-coded alerts on both the terminal interface and the full-stack web dashboard.

8.2 Pipeline Integrity and Training Inference Parity

A critical empirical lesson uncovered during practical deployment was the strict necessity of end-to-end transformation symmetry between offline training and live inference. In early deployment iterations, a feature transformation discrepancy was identified where discriminative feature weighting ($3.0\times$ for high-importance flow dynamics and $2.0\times$ for medium-importance attributes) was applied during estimator training but omitted in the live inference preprocessing step.

This discrepancy caused significant confidence skew on boundary attack signatures; specifically, probe and port-scanning vectors exhibited severely degraded classification confidence (collapsing

to borderline probabilities of $\sim 4\%$). Rectifying this pipeline bug by ensuring exact symmetry in RobustScaler standardization and weight multiplication immediately restored high-confidence detection (75%-96%) across live network flows. This underscored that high theoretical model accuracy remains fragile unless absolute parity is enforced between training-time and runtime feature engineering pipelines.

8.3 Domain Generalization and Continuous Adaptation

Evaluating the trained model on live network environments revealed a pronounced generalization gap between static academic benchmarks and modern operational attack tools:

- **TCP Stack and Tool Evolution:** Benchmark datasets recorded in simulated testbed environments capture tool behaviors and operating system TCP stacks specific to their recording era. When modern attack utilities (such as Go-based HTTP flooding scripts or updated Hydra brute-force engines) are executed against contemporary operating systems, deviations in TCP window negotiation, inter-arrival timing, and header lengths cause pure benchmark-trained models to struggle.
- **Syntactic Artifacts vs. Behavioral Signatures:** In real-time testing, the baseline model initially exhibited confusion between scanning probes and brute-force attempts because fixed tool payloads (such as Hydra's 56-byte FTP commands versus Nmap's 52-byte probe packets) occupied adjacent feature spaces. By capturing local traffic profiles, applying targeted labeling, and retraining the ensemble on localized behavioral patterns, the model successfully transitioned from memorizing tool-specific payload artifacts to recognizing generalized behavioral signatures (flow duration patterns, packet burst rates, and connection symmetries). This demonstrated that continuous learning and localized retraining loops are indispensable for real-world intrusion detection systems.

8.4 Critical Analysis of Operational Limitations

While the ensemble attained exceptional performance on the curated test set (surpassing 99.9% accuracy with an FPR of 0.021% on benign traffic), real-world deployment exposed critical, predictable edge cases inherent to flow-based tabular architectures:

- **The Single-Packet / Zero-Duration Trap (UDP Noise):** The 40 flow-level tabular features do not encode transport-layer protocol semantics (TCP versus UDP). Consequently, legitimate single-packet UDP interactions (such as Network Time Protocol (NTP) clock synchronizations characterized by 1 forward packet, 0 backward packets, and 0 duration) produce flow vectors statistically indistinguishable from single-probe reconnaissance scans. This occasionally triggered false-positive Port Scan alerts on benign background NTP traffic, demonstrating that

purely statistical flow models require complementary protocol-aware whitelisting or cross-flow correlation.

- **The Temporal Trap (Slow Benign Transactions):** Tree-based models heavily exploit Flow Duration to isolate sustained denial-of-service flooding. However, benign connections with long idle delays or slow server response times (such as slow HTTP/HTTPS transactions spanning 40 to 60 seconds with minimal packet volume) can mimic the temporal profile of low-and-slow DoS attacks (e.g., Slowloris or Hulk). This led to occasional borderline DoS false positives on sluggish web traffic, highlighting that models can become biased toward temporal duration over volumetric packet density.
- **Payload and Encryption Vulnerabilities:** While tabular models reliably identify cleartext protocol artifacts (such as predictable payload distributions in FTP brute-force attempts), identical credential attacks transported across encrypted channels (such as SSH) yield altered packet distributions and encrypted payloads that evade artifact-dependent decision trees.

8.5 Concluding Remarks

The ultimate takeaway of the NIDAS project is that benchmark test accuracy, while a vital developmental baseline, represents only a laboratory metric. The true measure of an operational intrusion detection system lies in identifying its operational breaking points under live conditions and understanding the root causes of misclassifications. Flow-level machine learning provides an exceptionally fast, high-throughput mechanism for intercepting volumetric floods and high-intensity exploits. However, achieving production-grade resilience demands a hybrid architecture that augments statistical tabular ensembles with protocol-level heuristic filters, multi-flow connection tracking, and continuous localized adaptation.

9. Future Work

While the NIDAS implementation establishes a complete and validated operational framework, several promising directions are identified to advance the system toward enterprise-scale cybersecurity deployment:

9.1 Continual Learning and Adaptive Concept Drift Mitigation

A persistent challenge in production intrusion detection is concept drift, where network traffic distributions evolve dynamically due to new software deployments, operating system updates, and evolving adversary tactics. In the current architecture, model retraining is conducted offline using curated historical partitions. Future iterations should incorporate streaming continual learning mechanisms, such as Hoeffding Adaptive Trees and experience-replay buffers, allowing the base estimators to assimilate incremental updates from validated live traffic without suffering from catastrophic forgetting. Furthermore, integrating statistical divergence monitors (such as the Population Stability Index (PSI) and Wasserstein distance computed over rolling windows of flow inter-arrival times) would enable automated triggering of localized retraining routines whenever significant distribution shifts are detected.

9.2 Explainable AI (XAI) and Real-Time SHAP Attributions

In operational Security Operations Centers (SOCs), automated alerts must be accompanied by transparent rationale to enable rapid triage and incident response. While the current dashboard surfaces calibrated class probabilities, integrating real-time Explainable AI (XAI) techniques represents a crucial enhancement. Implementing an optimized TreeSHAP (SHapley Additive exPlanations) engine within the inference pipeline would compute exact feature contribution vectors for every flagged connection. For instance, when a flow is classified as DoS, the interface could present an immediate local attribution waterfall chart indicating that elevated Flow Duration and extreme Bwd Packet Length Std contributed $+0.42$ and $+0.35$ respectively toward the malicious score. Optimizing TreeSHAP tree traversal through path memoization will ensure that explanatory attribution adds less than 1.5 milliseconds of latency to the alerting dispatch.

9.3 Hardware Acceleration and Kernel-Level Packet Inspection

The current Python-based CICFlowMeter operates in user space via standard network sockets, which delivers dependable throughput for monitoring standard local subnets but introduces operating system context-switching overhead under line-rate gigabit flooding. Future work will investigate offloading flow extraction directly into the Linux operating system kernel using eBPF (Extended Berkeley Packet Filter) and XDP (eXpress Data Path). By performing packet hashing,

timestamping, and counter accumulation within the driver layer before packets reach the OS networking stack, throughput can scale toward 10 Gbps and 100 Gbps enterprise backbones. Concurrently, converting the serialized ensemble into optimized Open Neural Network Exchange (ONNX) or TensorRT binaries will allow parallelized tree evaluation on dedicated edge inference accelerators.

9.4 Hybrid Unsupervised Zero-Day Threat Interception

The proposed ensemble excels at multi-class classification across known attack families (achieving 99.94% test accuracy). However, purely supervised learning remains inherently constrained by the scope of historical training labels. To protect against novel zero-day exploits and sophisticated advanced persistent threats (APTs), future iterations will couple the supervised soft-voting ensemble with an unsupervised anomaly detection branch, such as a deep convolutional Autoencoder or an Isolation Forest fitted exclusively on legitimate baseline traffic. When incoming flows exhibit low supervised confidence alongside high autoencoder reconstruction error, the system will flag the traffic as an uncharacterized zero-day anomaly, quarantining the connection and routing telemetry to human security analysts for forensic payload analysis.

10. Appendix

10.1 Minimum Software Requirements

To clone, deploy, and execute the intrusion detection system on a client computer for both live network traffic sniffing and offline PCAP analysis, the software environment, drivers, and execution privileges outlined in Table 10.1 must be satisfied. Because the deployment operates in inference mode using pre-trained model artifacts and serves a pre-compiled dashboard bundle, compiler toolchains, Node.js runtimes, and frontend build tools are completely unnecessary on the host system.

Component	Client Specification	Operational Scope and Configuration Details
Operating System	Windows 10 / 11 / Server 2016+; Linux (Ubuntu 20.04+, Debian 11+, Fedora, Arch, CentOS/RHEL); macOS 11+ (Big Sur or newer)	Full 64-bit architecture support across x86_64 and ARM processors (including Apple Silicon M-series).
Software and Runtime	Git (for repository cloning); CPython 3.10, 3.11, or 3.12 (64-bit)	Core execution environment. Node.js (v18+) is only needed if modifying frontend source files in frontend/src/; runtime serves pre-compiled frontend/dist/ directly.
Packet Capture Driver	Npcap (Windows); libpcap (Linux); libpcap / BPF (macOS)	Required strictly for live interface sniffing. Windows requires Npcap installed in WinPcap API-compatible mode. Linux installs via libpcap-dev or libpcap-devel. macOS provides built-in BPF.
Execution Privileges	Standard user for offline PCAP mode; Elevated / administrative rights for live capture mode	Offline PCAP processing requires no elevated rights. Live capture requires Run as Administrator on Windows, superuser (sudo) or packet capture capability (sudo setcap cap_net_raw,cap_net_admin=eip) on Linux, or /dev/bpf* access on macOS.
Operator Interface	Modern standards-compliant web browser (Chrome, Firefox, Edge, Brave, Safari)	Requires WebSocket protocol support for the live SOC dashboard telemetry stream served at port 8000.
Python Dependencies	scapy, pandas, joblib, scikit-learn, xgboost, numpy, fastapi, uvicorn, websockets	Installed via pip. Supports live packet capture, model unpickling (VotingClassifier, RobustScaler), XGBoost gradient boosting, matrix calculation, and WebSocket telemetry.

Table 10.1: Minimum Software Requirements

The Python dependency stack is installed via pip (pip install scapy pandas joblib scikit-learn xgboost numpy fastapi uvicorn websockets), providing scapy for packet capture and PCAP stream decoding; scikit-learn, xgboost, and joblib for ensemble model deserialization and inference; pandas and numpy for vectorized flow calculations; and fastapi, uvicorn, and websockets for the asynchronous REST backend and real-time dashboard stream.

10.2 Minimum Hardware Requirements

Because NIDAS operates strictly in inference mode during client deployment and utilizes a streaming, constant-memory flow processing engine, the hardware demands on the host machine are modest. The system performs lightweight CPU-based tree traversal without requiring dedicated hardware accelerators. Table 10.2 outlines the minimum and recommended client hardware specifications.

Component	Minimum	Recommended	Operational Notes
CPU	Dual-Core 64-bit x86 / ARM	Quad-Core 2.0+ GHz	Multi-core architecture benefits parallel batch PCAP processing (via the workers flag) and multi-threaded live sniffing.
RAM	2 GB available	4 GB – 8 GB system memory	Accommodates the pre-trained ensemble model (ensemble_model.pkl 25 MB), active flow state tables, pandas DataFrames, and FastAPI network buffers.
Disk Space	200 MB	1 GB – 2 GB	35 MB for repository and serialized models + 500 MB for Python libraries (xgboost, scikit-learn, scapy, pandas) + operating headroom for CSV outputs and PCAP dumps.
Network Card	Standard physical or virtual NIC	Promiscuous mode support	Required only for live packet capture on physical network interfaces or virtual adapters. Purely offline PCAP analysis requires no network adapter.
GPU	Not required	Not required	Client deployment executes purely CPU-based inference; dedicated GPUs or CUDA runtimes are completely unnecessary.

Table 10.2: Minimum Hardware Requirements

11. References

[1] Ness S, Eswarakrishnan V, Sridharan H, Shinde V, Janapareddy N V P, & Dhanawat V. (2025). A Robust Two-Tier Intrusion Detection System Using Machine Learning and Deep Learning Techniques. *IEEE Access*, vol. 13, pp. 11623–11639. Available from: https://doi.org/10.1109/ACCESS.2025.3526988

[2] Fathima AHN, Ibrahim SPS, Khraisat A. (2024). Enhancing Network Traffic Anomaly Detection: Leveraging Machine Learning and Feature Reduction for Effective Threat Detection. *IEEE Access*, vol. 12, pp. 131845–131859. Available from: https://doi.org/10.1109/ACCESS.2024.3458903

[3] Ji C, Liu H, Dai W. (2025). Hybrid Model for Network Traffic Anomaly Detection Based on Parallel Temporal Features and Residual Channel-Spatial Attention Mechanism. *IEEE Access*, vol. 13, pp. 26179–26190. Available from: https://doi.org/10.1109/ACCESS.2025.3538319

[4] Canadian Institute for Cybersecurity (CIC). (2017). Intrusion Detection Evaluation Dataset (CIC-IDS2017). University of New Brunswick. Available from: https://www.unb.ca/cic/datasets/ids-2017.html

[5] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.

[6] Chen T, Guestrin C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. Available from: https://doi.org/10.1145/2939672.2939785

[7] Sharafaldin I, Lashkari A H, Ghorbani A A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116. Communications Security Establishment (CSE) & Canadian Institute for Cybersecurity (CIC). Available from: https://www.unb.ca/cic/datasets/ids-2018.html

[8] Draper-Gil G, Lashkari A H, Mamun M S I, Ghorbani A A. (2016). Characterization of Encrypted Traffic with Comprehensive Features. In *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 407–414.